

Parallel & Concurrent Programming

For Rust Lab Books: pg. 2-49

For Rust Final Lab: pg. 45-49

For Cuda Lab Books: pg.49-90

For Cuda Final Lab: pg. 89-90

Lab B

Task 1: Q1. First threads

Solution:

```
mod my_function;
mod my_function2;
use my_function::my_function;
use my_function2::my_function2;

fn main() {
    std::thread::spawn(move || my_function() );
    std::thread::spawn(move || my_function2());
}

pub fn my_function() {
    std::thread::sleep(std::time::Duration::new(5, 0));
    println!("Hello, world!");
}

pub fn my_function2() {
    println!("This is the second function!");
}
```

Output:

Compiling first_thread v0.1.0 (C:\Users\omitc\source\repos\wjb-lab-b-Oliver-03-1-main\first_thread)

Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.65s

Running `target\debug\first\_thread.exe`

This is the second function!

* Terminal will be reused by tasks, press any key to close it.

Reflection:

I have set up 2 functions that print a line to the console and made the main function call them using 2 threads.

By adding a delay to one of the functions it means that it doesn't run as the main thread finishes before it and once the main thread is complete the project terminates.

When neither thread had a delay a race condition was created where either output could have been printed first. I could not get mine to have the outputs jumbled but it is possible.

Task 1: Q2. Joining threads

Solution:

```
mod my_function;
```

```
mod my_function2;
```

```
use std::thread::{self, JoinHandle};
```

```
fn main() {
```

```
    let num_of_threads: i32 = 5; // You can set this to any number
```

```
    let mut list_of_threads: Vec<thread::JoinHandle<()>> = vec!(); // Create a  
    mutable vector to store the thread handles
```

```
    for id in 0..num_of_threads {
```

```
        // Alternate between my_function and my_function2
```

```

let handle = if id % 2 == 0 {
    thread::spawn(move || {
        my_function::my_function();
    })
} else {
    thread::spawn(move || {
        my_function2::my_function2();
    })
};

list_of_threads.push(handle); // Store the thread handle in the list
}

// Join all threads to ensure they complete before the main thread exits
for handle in list_of_threads {
    handle.join().unwrap();
}

// Print message after all threads have terminated
println!("All threads have terminated.");
}

```

Output:

Hello World!

Hello Second World!

Hello World!

Hello Second World!

Hello World!

All threads have terminated.

Reflection:

This code creates a number of threads and a mutable list of thread handles. Mutable allows the variable to be changed as in rust they are always constant. This will allow the program to wait for a thread to complete using the .Join method later in the code.

The for loop just assigns the thread to either function depending on whether it's an even or odd number. Then the join method is called on the threads handle so that the program will wait for the thread to finish before another thread gets to go.

The unwrap will return an error if the thread panics. Once all threads are completed the final line is printed.

Task 1: Q3. Experimentation

Solution:

```
pub fn my_function(id: usize) {
    let sum: u64 = (1..1_000_000).sum();
    println!("Thread {}: The sum is {}", id, sum);
}

pub fn my_function2(id: usize) {
    // Simulate some work by sorting a large vector
    let mut vec: Vec<u64> = (1..1_000_000).rev().collect();
    vec.sort();
    println!("Thread {}: Finished sorting", id);
}
```

Output:

Thread 0: The sum is 499999500000

Thread 4: The sum is 499999500000

Thread 8: The sum is 499999500000

Thread 2: The sum is 499999500000

Thread 6: The sum is 499999500000

Thread 5: Finished sorting

Thread 9: Finished sorting

Thread 1: Finished sorting

Thread 3: Finished sorting

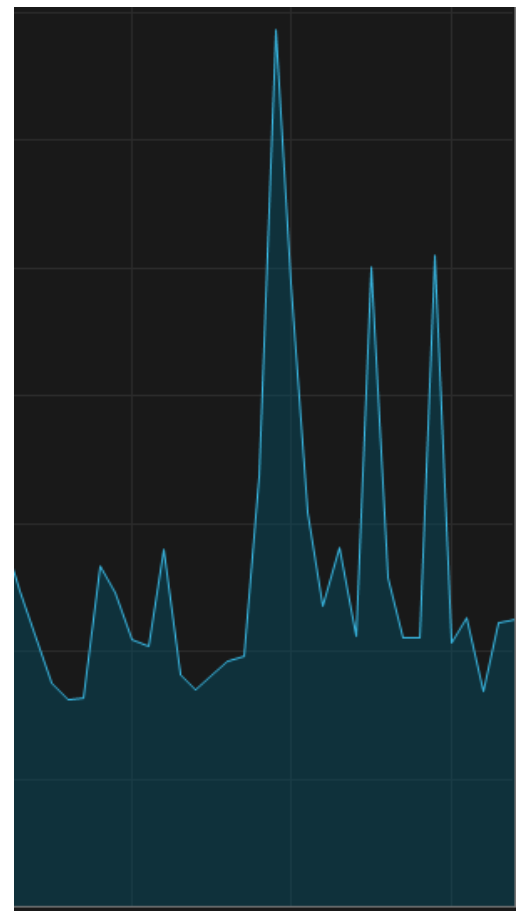
Thread 7: Finished sorting

Reflection:

I got copilot to think of two tasks it implemented a function that adds the sum of every number between 1 and 999,999 and a function that reverses every number between 1 and 999,999 so it would then be 999,999 to 1.

As you can see in the output the threads finished in different orders this shows that the data was passed over multiple times as it could not be completed in the one timestep. The sorting function was more intensive than the sum function as they all finish after the even numbers.

I then added 100 threads to do these tasks to see the impact in task manager as 10 threads was not making a noticeable difference. As you can see this caused small spikes in the CPU as the tasks were completing. I ran the code 3 times hence the three peaks. The first time I run the code on the CPU is always



the longest because it needs to compile any changes. Another thing I noticed when I changed the threads to 100 is that all of the tasks finished at different times whilst most of the sum tasks were finished first there were some that finished after sorting tasks. This is likely due to the CPU prioritizing other tasks that we didn't get to see when there were a shorter number of threads.

Lab Book C

Task 1:

Task 2: Ownership Part 1

Solution:

```
fn main() {  
    let p1 = Person::new("Jane", 30);  
  
    print_person(&p1);  
    print_person(&p1);  
}  
  
fn print_person(p: &Person) {  
    println!("{}", p.name, p.age);  
}
```

Output:

Jane is 30 years old

Jane is 30 years old

Reflection:

Due to Rusts ownership rules we must use a reference to person rather than taking ownership of it. This now means that we are borrowing the object and therefore can have multiple references to person objects.

Task 3: Ownership Part 2

Solution:

```
fn main() {  
    let mut p1 = Person::new("Jane", 30);  
  
    let r1 = & p1;  
    let r2 = & p1;  
  
    print_person(r1);  
    print_person(r2);  
  
    let r3 = &mut p1;  
  
    increment_age(r3);  
  
    print_person(r3);  
}  
  
fn print_person(p: & Person) {  
    println!("{}", p.name, p.age);  
}  
  
fn increment_age(p: &mut Person) {  
    p.age = p.age + 1;  
}
```

Output:

Jane is 30 years old

Jane is 30 years old

Jane is 31 years old

Reflection:

Mutable references allow the value of something to be modified immutable does not. Rust only allows one mutable reference to an object at a time. If I created a mutable r4 person below r3 it would cause an error. Likewise, you cannot have a mutable reference while there is an immutable reference to an object in scope. The reason why this code works is because the immutable references are out of scope as the references are no longer being used so the mutable reference is allowed to be created after.

Task 4: Familiarise yourself with Rusts Classes**Solution:**

```
pub struct SharedData {  
    value: u32  
}
```

```
impl SharedData {  
    pub fn new() -> SharedData {  
        SharedData {  
            value: 0  
        }  
    }  
}
```

```
pub fn update(&mut self) {  
    let local_value = self.value;  
    std::thread::sleep(std::time::Duration::new(1,0));  
    self.value = local_value + 1;  
}
```

```
pub fn print(&self) {  
    println!("SharedData: value = {}", self.value)  
}
```

```
}
```

Reflection:

Data Members: The struct part of the code is where the data members are defined.

The Constructor: The new method is the constructor, and it creates an instance of the struct.

Methods: Methods are created with fn (function) pub(public) is the visibility. If pub isn't added then its private by default. The update method takes a mutable reference to the instance of the struct allowing the data inside to be changed. Then the print method takes an immutable reference to the instance of the struct so it can print the data.

Any methods that are associated with the struct (data members) must be defined in the implementation (impl) block.

The self-object: The self-object is just a reference to the instance of the struct much like the this keyword in c#.

Task 5: Create a thread function for shared data

Solution:

```
mod shared_data;
```

```
use shared_data::SharedData;
```

```
use std::thread;
```

```
fn main() {
```

```
    let mut data = SharedData::new();
```

```
    let handle = thread::spawn(move || {
```

```
        thread_function(&mut data);
```

```
    });
```

```
    handle.join().unwrap();
```

```
}
```

```
fn thread_function(data: &mut SharedData) {  
    data.update();  
    data.print();  
}
```

Output:

SharedData: value = 1

Reflection:

In the working version an instance of shared data called data is created. Then a thread is created which moves the ownership of data from the main thread to the new thread. A mutable reference to data is then is passed to the thread function where the methods are called. When you move the print method into the main thread the data is no longer in the main threads scope to be printed therefore the print method has no data to print.

Lab D

Ownership Limitations:

Solution:

```
struct Aircraft<'a> {  
    name: String,  
    engines: Vec<&'a Engine<'a>>,}  
  
impl Aircraft<'_> {  
    pub fn new(name_param: &str) -> Aircraft {  
        Aircraft {  
            name: name_param.to_string(),  
            engines: Vec::new()  
        }  
    }  
}  
  
struct Engine<'a> {  
    name: String,  
    aircraft: &'a Aircraft<'a>,  
}  
  
impl<'a> Engine<'a> {  
    pub fn new(name: &str, aircraft: &'a Aircraft<'a>) -> Self {  
        Self {  
            name: name.to_string(),  
            aircraft,  
        }  
    }  
}
```

```

    }
}

fn main() {
    let mut f18 = Aircraft::new( "F-18" );
    let engine1 = Engine::new( "General Electric F404",&f18 );
    let engine2 = Engine::new( "General Electric F404",&f18 );

    f18.engines.push (&engine1);
    f18.engines.push (&engine2);

    println! ("Aircraft: {} has a {} and {} ", f18.name, f18.engines[0].name,
f18.engines[1].name );
}

```

Output: Error Mutable borrowing.

Reflection:

Lifetime Parameters ensure that references within a struct live at least if the struct itself does. Without these, you risk having dangling references.

Trying to expand this code the way I have by adding a reference to the engine to link the aircraft causes a mutable borrowing error. This is because you need the aircraft to be mutable to add the engines but you need immutable references from the engines to the aircraft.

Reference counters:

Solution:

use std::rc::Rc;

```

struct Aircraft {
    name: String,
    engines: Vec<Rc<Engine>>,
}

```

```
impl Aircraft {  
    pub fn new(name_param: &str) -> Aircraft {  
        Aircraft {  
            name: name_param.to_string(),  
            engines: Vec::new()  
        }  
    }  
}
```

```
struct Engine {  
    name: String,  
    requires_service: bool, // New data member  
}
```

```
impl Engine {  
    pub fn new(name_param: &str) -> Engine {  
        Engine {  
            name: name_param.to_string(),  
            requires_service: true,  
        }  
    }  
}
```

```
    pub fn service(&mut self) {  
        self.requires_service = false;  
    }  
}
```

```
fn main() {
```

```

let engine1 = Rc::new(Engine::new("General Electric F404"));
let engine2 = Rc::new(Engine::new("General Electric F404"));

let mut f18 = Aircraft::new("F-18");

f18.engines.push(engine1);
f18.engines.push(engine2.clone());

println!(
    "Aircraft: {} has a {} and {}",
    f18.name, f18.engines[0].name, f18.engines[1].name
);
println!("Engine: {}", f18.engines[0].name);
println!("Engine: {}", f18.engines[1].name);

// Create a third engine and service it
let mut engine3 = Engine::new("General Electric F404");
engine3.service();
println!("Engine3 requires service: {}", engine3.requires_service);

engine2.service();
}

```

Reflection:

Reference counters allow multiple ownership through a counter of the number of references that data currently has. When the counter is 0 the data is cleaned up. `Rc<Engine>` wraps the Engine instances to enable multiple ownership. Each time you clone an `Rc`, it increments the reference count. This allows both Aircraft and other parts of the program to own references to the same Engine instances.

When the Clone is removed from the code ownership is passed from the main thread to the aircraft struct this means when the engine names are later needed they cannot be

used. If clone was used a reference is created and that is sent to the aircraft so you can access engine later if needed.

When you do the service part. There is an error with engine 2 because by default rc make all data immutable as it is read only.

Lab E

Q1. Thread safe printing

Solution

```
use std::sync::{Arc, Mutex};

use std::thread;

fn main() {
    let num_of_threads = 4;

    let print_lock = Arc::new(Mutex::new(())); // Shared lock
    let mut array_of_threads = vec![];

    for id in 0..num_of_threads {
        let print_lock = Arc::clone(&print_lock);
        array_of_threads.push(thread::spawn(move || {
            let _guard = print_lock.lock().unwrap(); // The lock line
            print_lots(id);
        }));
    }

    for t in array_of_threads {
        t.join().expect("Thread join failure");
    }
}

fn print_lots(id: u32) {
    println!("Begin [{}]", id);
    for _i in 0..100 {
```

```

        print!("{}", id);
    }
    println!("\nEnd [{}]", id);
}

```

Reflection:

The mutex is a lock that makes the threads wait their turn before they begin doing a task. Arc is an atomic reference counted pointer which allows for multi threaded references where rc can't. Each thread then instead gets a clone of the print lock. They will then try and access the lock but if another thread is using the lock it must wait.

I thought that if the mutex fails to release the other threads cannot access the lock and won't be able to do what they want to do. However, when I added a infinite loop to the code the other threads worked as expected I can only assume that rust has some kind of pre-built functionality that stops this.

```

for id in 0..num_of_threads {
    let print_lock = Arc::clone(&print_lock);
    array_of_threads.push(thread::spawn(move || {
        let guard = print_lock.lock().unwrap(); // Lock acquired
        loop {
        }
        print_lots(id);
    }));
}

```

Q2. Triangles using OpenGL

Solution:

```

for i in 0 .. 10 {
    let x = rand::random::<f32>();

    // Calculate the position of the triangle
    let pos_x : f32 = delta_t + ((i as f32) * x);
    let pos_y : f32 = delta_t + ((i as f32) * x);
}

```

```
let pos_z : f32 = 0.0;
```

Reflection:

$((i \text{ as } f32) * 0.1)$ is what causes the triangles to be spread out over certain distances by changing it to a random int x it causes the distances between the 10 triangles to change.

Lab F P&C

Question 1:

Solution:

```
use rand::Rng;
```

```
use std::thread;
```

```
use std::time::{Duration, Instant};
```

```
const PARTICLE_COUNT: usize = 100;
```

```
const ENCLOSURE_SIZE: f32 = 10.0;
```

```
const NUM_OF_THREADS: usize = 4;
```

```
#[derive(Debug, Copy, Clone)]
```

```
pub struct Particle {
```

```
    x: f32,
```

```
    y: f32,
```

```
}
```

```
impl Particle {
```

```
    fn new(x: f32, y: f32) -> Self {
```

```
        Self { x, y }
```

```
    }
```

```
    fn move_randomly(&mut self) {
```

```
        let mut rng = rand::thread_rng();
```

```
        let dx = (rng.gen::<f32>() - 0.5) * 2.0; // Random step between -1.0 and 1.0
```

```
        let dy = (rng.gen::<f32>() - 0.5) * 2.0;
```

```

    self.x = (self.x + dx).clamp(0.0, ENCLOSURE_SIZE);
    self.y = (self.y + dy).clamp(0.0, ENCLOSURE_SIZE);
}
}

struct ParticleSystem {
    particles: Vec<Particle>,
}

impl ParticleSystem {
    fn new() -> Self {
        let mut rng = rand::thread_rng();
        let mut particles = Vec::with_capacity(PARTICLE_COUNT);

        for _ in 0..PARTICLE_COUNT {
            let x = rng.gen_range(0.0..ENCLOSURE_SIZE);
            let y = rng.gen_range(0.0..ENCLOSURE_SIZE);
            particles.push(Particle::new(x, y));
        }

        Self { particles }
    }

    fn display_particles(&self) {
        for particle in &self.particles {
            println!("Particle at {:.2}, {:.2}", particle.x, particle.y);
        }
    }
}

```

```
fn move_particles(&mut self) {
    for particle in &mut self.particles {
        particle.move_randomly();
    }
}
```

```
fn simulate_movement(&mut self, duration_secs: u64) {
    let start_time = Instant::now();
    while start_time.elapsed().as_secs() < duration_secs {
        self.move_particles();
        self.display_particles_grid(); // Show grid instead of raw positions
        thread::sleep(Duration::from_millis(500));
    }
}
```

```
fn display_particles_grid(&self) {
    let mut grid = vec![vec!['.', 11]; 11]; // 11x11 grid (0-10 range)

    for particle in &self.particles {
        let x = particle.x.round() as usize;
        let y = particle.y.round() as usize;
        grid[y][x] = '*'; // Mark particle position
    }

    for row in grid.iter().rev() {
        println!("{}", row.iter().collect::<String>());
    }

    println!("{}", "-----"); // Separator
}
```

```

    }

}

fn main() {
    let mut particle_system = ParticleSystem::new();

    println!("Initial Positions:");
    particle_system.display_particles();

    particle_system.move_particles();
    println!("\nAfter Moving:");
    particle_system.display_particles();

    println!("\nSimulating movement for 10 seconds...");
    particle_system.simulate_movement(10);
}

```

Output:

Initial Positions:

Particle at (6.43, 3.72)

Particle at (2.17, 4.18)

Particle at (5.24, 8.36)

Particle at (7.22, 3.23) etc...

After Moving:

Particle at (7.24, 4.12)

Particle at (3.10, 4.03)

Particle at (5.27, 9.23)

Particle at (6.98, 3.87) etc...

Simulating movement for 10 seconds...

```

..* *****
...

..*** *
...

*****
..

.*****.
.

...***.
.

.***.
.

.***.
.

.*****
.

** *****
..

** .***
. ....

*****
.

-----

..* **
...

..* ****
...

******
.

*****.
.

*****
..

*****
.

*****
....

*****
.

*****
..

** * ***
..

*****
.

...*****
..

```

Reflection:

The particles are created and placed randomly in a grid. This is done by using clamping which ensures that the particles stay within the boundaries of the grid and the randRNG where a float between 0 and 1 will be created. By subtracting 0.5 and multiplying by two we change the random value being created to -1 and 1 allowing for particle to move left right up or down randomly.

For the initial placement of the particles and the first movement of the particles the coordinates are shown in the terminal so you can see if they were randomly moved or if

they were out of bounds. When the simulation of the particles is being done their positions in the grid are outputted instead. This is done by rounding their positions to the nearest whole number then placing them in the corresponding element in the grid which means some particles will occupy the same space so you can't see how many particles are in the same space.

[derive(Debug, Copy, Clone)]: Debug: Enables println!("{:?}", particle). Copy: Allows copying values without moving ownership. Clone: Allows explicit cloning with .clone().

```
#[test]
```

```
fn test_particle_movement() {  
    let mut particle = Particle::new(5.0, 5.0);  
  
    let original_x = particle.x;  
    let original_y = particle.y;  
  
    particle.move_randomly();  
  
    assert!(particle.x >= 0.0 && particle.x <= ENCLOSURE_SIZE);  
    assert!(particle.y >= 0.0 && particle.y <= ENCLOSURE_SIZE);  
    assert!(particle.x != original_x || particle.y != original_y);  
}
```

```
#[test]
```

```
fn test_particle_system_movement() {  
    let mut particle_system = ParticleSystem::new();  
  
    let original_positions: Vec<(f32, f32)> = particle_system  
        .particles  
        .iter()  
        .map(|p| (p.x, p.y))  
        .collect();  
  
    particle_system.move_particles();
```

```

    let mut moved = false;
    for (i, particle) in particle_system.particles.iter().enumerate() {
        if particle.x != original_positions[i].0 || particle.y != original_positions[i].1 {
            moved = true;
            break;
        }
    }

    assert!(moved, "At least one particle should have moved.");
}
}

```

Output

running 3 tests

test tests::test_particle_movement ... ok

test tests::test_particle_initialization ... ok

test tests::test_particle_system_movement ... ok

successes:

successes:

tests::test_particle_initialization

tests::test_particle_movement

tests::test_particle_system_movement

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

Reflection:

I learned how to create unit tests in rust. The tests check that all particles made are within the bounds of the grid and that the movement methods of both particles actually work.

Question 2:

Solution:

```
use rand::Rng;
use std::thread;
use std::time::{Duration, Instant};
use scoped_threadpool::Pool; // Import thread pool
```

```
const PARTICLE_COUNT: usize = 500;
const ENCLOSURE_SIZE: f32 = 10.0;
const NUM_OF_THREADS: usize = 4;
```

```
#[derive(Debug, Copy, Clone)]
```

```
pub struct Particle {
```

```
    x: f32,
```

```
    y: f32,
```

```
}
```

```
impl Particle {
```

```
    fn new(x: f32, y: f32) -> Self {
```

```
        Self { x, y }
```

```
    }
```

```
    fn move_randomly(&mut self) {
```

```
        let mut rng = rand::thread_rng(); // Updated
```

```
        let dx = (rng.gen::<<f32>>() - 0.5) * 2.0; // Updated
```

```
        let dy = (rng.gen::<<f32>>() - 0.5) * 2.0;
```

```
        self.x = (self.x + dx).clamp(0.0, ENCLOSURE_SIZE);
```

```
        self.y = (self.y + dy).clamp(0.0, ENCLOSURE_SIZE);
```

```
    }
```

```
}
```

```
struct ParticleSystem {  
    particles: Vec<Particle>,  
}
```

```
impl ParticleSystem {  
    fn new() -> Self {  
        let mut rng = rand::thread_rng(); // Updated  
        let mut particles = Vec::with_capacity(PARTICLE_COUNT);  
  
        for _ in 0..PARTICLE_COUNT {  
            let x = rng.gen_range(0.0..ENCLOSURE_SIZE); // Updated  
            let y = rng.gen_range(0.0..ENCLOSURE_SIZE); // Updated  
            particles.push(Particle::new(x, y));  
        }  
  
        Self { particles }  
    }  
}
```

```
fn move_particles_threaded(&mut self) {  
    let mut pool = Pool::new(NUM_OF_THREADS as u32);  
  
    // Store the length in a separate variable to avoid borrowing self.particles immutably  
    // later.  
    let total = self.particles.len();  
    let mut chunks: Vec<&mut [Particle]> = self.particles.chunks_mut(total /  
    NUM_OF_THREADS).collect();  
  
    pool.scoped(|scope| {
```

```

    for chunk in chunks.iter_mut() {
        scope.execute(move || {
            for particle in chunk.iter_mut() {
                particle.move_randomly();
            }
        });
    }
}); // All threads are joined at the end of the scope.
}

```

```

fn display_particles(&self) {
    for particle in &self.particles {
        println!("Particle at ( {:.2}, {:.2} )", particle.x, particle.y);
    }
}

```

```

fn simulate_movement(&mut self, duration_secs: u64) {
    let start_time = Instant::now();
    while start_time.elapsed().as_secs() < duration_secs {
        self.move_particles_threaded();
        self.display_particles_grid();
        thread::sleep(Duration::from_millis(500)); // Slow it down
    }
}

```

```

fn display_particles_grid(&self) {
    let mut grid = vec![vec!['.', 11]; 11]; // 11x11 grid (0-10 range)

    for particle in &self.particles {

```

```

        let x = particle.x.round() as usize;
        let y = particle.y.round() as usize;
        grid[y][x] = '*'; // Mark particle position
    }

    for row in grid.iter().rev() {
        println!("{}", row.iter().collect::<String>());
    }
    println!("-----");
}
}

fn main() {
    let mut particle_system = ParticleSystem::new();

    println!("Initial Positions:");
    particle_system.display_particles();

    println!("\nSimulating movement for 10 seconds with threads...");
    particle_system.simulate_movement(10);
}

```

Output:

Threaded performance

Debug Execution time: 10.11s

Debug Total Moves: 2519500

Release Execution time: 10.10s

Release Total Moves: 2889500

Non threaded performance:

Debug Total Execution Time: 10.05s

Total Moves: 2292500

Release Total Execution Time: 10.04s

Total Moves: 3452000

Reflection:

The main difference is in the move particle method. In this method I created a thread pool and split the particles into four chunks. Then the particle moving happens in parallel with each chunk updating at the same time. Due to scoped thread pooling once the thread has finished with its chunk it will wait for the others to finish.

Thread pool works by having a set allocation of threads that are tasks. If there are too many tasks then the tasks wait in a queue. When the task is done the thread moves onto the next task. In scoped pools which is what is used in the solution the program waits for all the tasks to be finished before moving on. This means that you don't need to have a join at the end as well. Scoped pools avoid many ownership issues and reduce overheads like copying, creating arc mutex locks and it makes sure all the threads are terminated once finished. In my solution the particles are mutable inside the scope and are changed once then the borrow is dropped when we go out of the scope so the next time we loop around we can change the particles again.

With the performance the non-threaded program was faster than the threaded program in execution time. I can only assume that the overhead in managing the threads was actually detrimental in a task as simple as this one. Then I decided to remove the sleep on both in the simulation and add a counter to the number of movements assuming that the faster the program runs the more updates it will be able to do. The results showed that this was partially correct as both of the threaded performances were better than the debugged non threaded performance however the release non threaded performed a lot better than even the release version of the threaded program. I once again assume that this is due to the managing of threads that was slowing the program. In a more complex task, the thread will be better

Lab G

Exercise 1:

Solution:

```
impl Particle {  
    fn new(x: f32, y: f32) -> Self {  
        Self { x, y }  
    }  
  
    fn move_randomly(&mut self) {  
        let mut rng = rand::thread_rng();  
        let dx = (rng.gen::() - 0.5) * 2.0;  
        let dy = (rng.gen::() - 0.5) * 2.0;  
        self.x = (self.x + dx).clamp(0.0, ENCLOSURE_SIZE);  
        self.y = (self.y + dy).clamp(0.0, ENCLOSURE_SIZE);  
    }  
  
    fn collide(&self, other: &Particle) -> bool {  
        let dx = self.x - other.x;  
        let dy = self.y - other.y;  
        let distance = (dx * dx + dy * dy).sqrt();  
        distance < COLLISION_THRESHOLD  
    }  
}  
  
fn detect_collisions(&self) -> usize {  
    let mut collisions = 0;  
    let n = self.particles.len();  
    for i in 0..n {
```

```

    for j in (i + 1)..n {
        if self.particles[i].collide(&self.particles[j]) {
            collisions += 1;
        }
    }
}

collisions
}

fn simulate_movement(&mut self, duration_secs: u64) {
    let start_time = Instant::now();
    while start_time.elapsed().as_secs() < duration_secs {
        self.move_particles_threaded();
        self.display_particles_grid();

        // Collision detection using a separate thread pool with 1 thread.
        let collision_count = {
            let mut collision_pool = Pool::new(1);
            let mut count = 0;
            collision_pool.scoped(|scope| {
                scope.execute(|| {
                    count = self.detect_collisions();
                });
            });
            count
        };
        println!("Collision count: {}", collision_count);
        thread::sleep(Duration::from_millis(500)); // Delay between iterations.
    }
}

```

Reflection:

In my code no locking is required as long as you don't increase the number of threads from 1. Currently there is no simultaneous reading as collision detection happens once the movements have happened as we use a scoped pool. Currently there are no race conditions either as there is no simultaneous reading and writing. If I changed the number of threads to check collisions in parallel or decided to run both movement and collisions concurrently then there would be issues. To optimise the code, I could use a spatial data structure to check for collisions as currently I'm just checking every pair of particles. I could also make collision detection calculations run in parallel which would require locking.

Exercise 2:

```
struct ParticleSystem {  
    particles: Vec<Particle>,  
    collision_count: AtomicUsize,  
}  
  
fn detect_collisions_atomic(&self) -> usize {  
    self.collision_count.store(0, Ordering::SeqCst);  
    let n = self.particles.len();  
    let mut pool = Pool::new(NUM_OF_THREADS as u32);  
    let chunk_size = (n + NUM_OF_THREADS - 1) / NUM_OF_THREADS;  
    pool.scoped(|scope| {  
        for chunk_start in (0..n).step_by(chunk_size) {  
            let chunk_end = (chunk_start + chunk_size).min(n);  
            let collision_counter = &self.collision_count;  
            let particles = &self.particles;  
            scope.execute(move || {  
                for i in chunk_start..chunk_end {  
                    for j in (i + 1)..n {  
                        if particles[i].collide(&particles[j]) {
```

```

        collision_counter.fetch_add(1, Ordering::SeqCst);
    }
}
}
});
}
});
self.collision_count.load(Ordering::SeqCst)
}

```

Reflection:

By using an atomic counter, we can use multiple threads for the collision detection logic. In my code we create the counter and set it to 0 and then create a thread pool of 4 threads. Then the threads are given four equal chunks and then they are scoped. The atomic counter will not let more than one thread write to it at a time, so it is safe from race conditions. The fetch add method increments the counter. The `Ordering::SeqCst` makes atomic updates behave as one global sequence ensuring that every increment is seen in the same order by all threads, and thus preventing bugs.

```

impl ParticleSystem {
    fn new() -> Self {
        let mut rng = rand::thread_rng();
        let mut particles = Vec::with_capacity(PARTICLE_COUNT);
        for _ in 0..PARTICLE_COUNT {
            let x = rng.gen_range(0.0..ENCLOSURE_SIZE);
            let y = rng.gen_range(0.0..ENCLOSURE_SIZE);
            particles.push(Particle::new(x, y));
        }
        Self {
            particles,
            collision_count: AtomicUsize::new(0)
        }
    }
}

```

```
}
```

```
fn detect_collisions(&self) -> usize {  
    let n = self.particles.len();  
    let mut collisions = 0;  
    for i in 0..n {  
        for j in (i + 1)..n {  
            if self.particles[i].collide(&self.particles[j]) {  
                collisions += 1;  
            }  
        }  
    }  
    collisions  
}
```

```
fn detect_collisions_atomic(&self) -> usize {  
    // Reset the counter  
    self.collision_count.store(0, Ordering::SeqCst);  
    let n = self.particles.len();  
    let mut pool = Pool::new(NUM_OF_THREADS as u32);  
    let chunk_size = (n + NUM_OF_THREADS - 1) / NUM_OF_THREADS;  
    pool.scoped(|scope| {  
        for chunk_start in (0..n).step_by(chunk_size) {  
            let chunk_end = (chunk_start + chunk_size).min(n);  
            let collision_counter = &self.collision_count;  
            let particles = &self.particles;  
            scope.execute(move || {  
                for i in chunk_start..chunk_end {  
                    for j in (i + 1)..n {  
                        if particles[i].collide(&particles[j]) {  
                            collision_counter.fetch_add(1, Ordering::SeqCst);  
                        }  
                    }  
                }  
            })  
        }  
    })  
}
```

```

        }
    }
}
});

}

});

self.collision_count.load(Ordering::SeqCst)

```

Reflection:

I created a new system that randomly places the particles and then calls both versions of the collision detection. It is then timed as to how quickly the an answer is given. I also double checked that the order of the method being called was not effecting things at all.

Non threaded called first:

Non-threaded collision detection: count = 26, time = 267.9µs Threaded collision detection: count = 26, time = 398.8µs

Non threaded called second:

Threaded collision detection: count = 37, time = 420.8µs

Non-threaded collision detection: count = 37, time = 271.5µs

This is because of the overhead required with threading. We would need a much larger particle size for the benefits to be seen.

Lab H

Exercise 1: Producer Consumer

Solution:

```
use std::sync::{Arc, Mutex, Condvar};
```

```
use std::thread;
```

```
use std::time::Duration;
```

```
struct SharedData {
```

```
    value: i32,
```

```
    version: u32,
```

```
    available: bool,
```

```
}
```

```
struct SharedState {
```

```
    data: Mutex<SharedData>,
```

```
    condvar: Condvar,
```

```
}
```

```
impl SharedState {
```

```
    fn new() -> Self {
```

```
        SharedState {
```

```
            data: Mutex::new(SharedData {
```

```
                value: 0,
```

```
                version: 0,
```

```
                available: false,
```

```
            }},
```

```
            condvar: Condvar::new(),
```

```
        }
```

```
    }
```

```

fn update(&self, new_value: i32) {
    let mut data = self.data.lock().unwrap();
    while data.available {
        data = self.condvar.wait(data).unwrap();
    }
    data.value = new_value;
    data.version += 1;
    data.available = true;
    self.condvar.notify_all();
}

fn wait_for_update(&self, last_version: u32) -> (i32, u32) {
    let mut data = self.data.lock().unwrap();
    while !data.available || data.version == last_version {
        data = self.condvar.wait(data).unwrap();
    }
    let value = data.value;
    let new_version = data.version;
    data.available = false;
    self.condvar.notify_all();
    (value, new_version)
}

fn main() {
    let shared_state = Arc::new(SharedState::new());

    {
        let state = shared_state.clone();
        thread::spawn(move || {

```

```

    for i in 1..=5 {
        println!("Producer: updating value to {}", i);
        state.update(i);
        thread::sleep(Duration::from_millis(200));
    }
});
}

{
    let state = shared_state.clone();
    thread::spawn(move || {
        let mut last_version = 0;
        for _ in 1..=5 {
            let (value, version) = state.wait_for_update(last_version);
            println!("Consumer: received new value {} with version {}", value, version);
            last_version = version;
            thread::sleep(Duration::from_millis(100));
        }
    });
}

thread::sleep(Duration::from_secs(2));
}

```

Output:

```

Producer 0: updating value to 1
Consumer 0: received new value 1 with version 1
Producer 1: updating value to 1
Consumer 2: received new value 1 with version 2
Producer 2: updating value to 1
Consumer 1: received new value 1 with version 3
Producer 0: updating value to 2

```

Consumer 1: received new value 2 with version 4

Producer 2: updating value to 2

Consumer 2: received new value 2 with version 5

Producer 1: updating value to 2

Consumer 0: received new value 2 with version 6

Producer 0: updating value to 3

Consumer 2: received new value 3 with version 7

Producer 1: updating value to 3

Consumer 1: received new value 3 with version 8

Producer 2: updating value to 3

Consumer 0: received new value 3 with version 9

Producer 0: updating value to 4

Consumer 1: received new value 4 with version 10

Producer 1: updating value to 4

Consumer 2: received new value 4 with version 11

Producer 2: updating value to 4

Consumer 0: received new value 4 with version 12

Producer 0: updating value to 5

Consumer 2: received new value 5 with version 13

Producer 1: updating value to 5

Consumer 1: received new value 5 with version 14

Producer 2: updating value to 5

Consumer 0: received new value 5 with version 15

Reflection: In this solution we have one piece of data that the producers are writing to. What happens is the producer will first check if the data has been consumed (via the flag) if not it will wait. If it has been it will take the data lock it and change its value. Then it will notify the consumers. The consumers will have a value called last version. If they see they are equal it means no new data has been made so they will wait. When the last data is no longer equal to the data they will consume the data and make the last data equal to the new data.

The bottle necks of this system are: The shared lock, limited buffer size and use of thread pools. The shared lock means that all the threads compete for the same lock and will all block each other which will be quite bad at scale. Limited buffer size means that the producers and consumers must take it in turns reading and writing the data. Instead, the data should be able to be stored in a queue so it can take multiple values from the producers and then the consumers can just take them as and when. Finally a thread pool could be used to change the role of a thread for example if we made the queue and the producers were producing at a rate faster than the consumers could consume we could take a producer thread and make it a consumer.

Lab I

Flocking File:

There is a bird struct that randomly gives the birds position and velocity vectors, and an acceleration vector set to zero. Then it has an update method in it that takes a mutable reference to itself and to the flock as well as the change in time and then calls another update method. This method calls the methods that compute the alignment, cohesion and separation forces as well as another force I made called the boundary force. I made the boundary force as it keeps the birds mostly inside the view of the screen.

All the methods use a has neighbours' method which checks its position with every other bird in the flock using a distance squared check to save on resources if the birds are within X distance to the current bird then they will be added to its neighbours list for calculations.

Separation: This force ensures that the birds don't crowd each other. It works by calculating the vector pointing away from each nearby bird and summing these vectors. The more birds in the vicinity, the stronger the separation force becomes, which prevents overlap and maintains personal space for each bird. The force is normalized and adjusted based on the number of neighbours to keep the birds from moving too erratically.

Alignment: This force causes the bird to match the velocity of its neighbours. It works by averaging the velocities of the birds in its neighbourhood and applying a force that aligns the bird's velocity with that average. This helps keep the flock moving cohesively, avoiding random movements that would disrupt the group behaviour.

Cohesion: Cohesion pulls the bird towards the centre of its neighbouring birds, encouraging the bird to stay close to the rest of the flock. The method calculates the average position of the neighbours and applies a force that draws the bird toward that centre. This keeps the flock together and moving as a unit.

Boundary Force: To keep the flock from moving off the screen, I introduced the boundary force. This force pulls birds back toward the centre if they get too far away from the predefined boundary radius. I applied a stronger force when birds exceed the boundary radius and a weaker one when they get closer to the edge. This ensures the birds remain within a visible area while still allowing some natural movement near the boundary.

Together, these forces are weighted by coefficients (w_s , w_a , w_c) to balance their influence on the bird's movement. The boundary force is also factored in to prevent the birds from straying too far from the centre. These forces are applied sequentially to the bird's acceleration, which then updates its velocity and position based on the time step (dt). This structure creates dynamic and coherent flock behaviour where the birds move naturally while maintaining formation.

Chunk Thread Locks:

The main goal was to utilize multiple threads by dividing the work into chunks to process different parts of the flock in parallel.

The first step was to create two clones of flock. They both use arc but one uses a mutex as well. The one that doesn't have a lock is used by the thread to read other flock data this is read only so it doesn't need to be locked as the threads accessing it concurrently won't cause any issues. The flock uses a mutex because it will be used to edit data about the birds.

Next, I calculate how many chunks the flock should be divided into. This is based on the total number of birds and the specified chunk_size. This ensures that each thread works on a smaller, manageable subset of birds, rather than all birds at once, which helps distribute the workload across available threads.

Then, for each chunk of birds, a new thread is spawned. The thread locks the shared flock and processes its respective chunk by calling the update method for each bird within that chunk. After the thread finishes updating its chunk, it joins back into the main thread to ensure synchronization. This way, all threads work concurrently and then combine the results back into the original flock.

Once all threads finish their work, the main thread locks the flock again to update it with the latest changes from all threads. This ensures the original flock is fully updated with the new bird positions and behaviours.

Thread Per Bird

In the thread-per-bird model, each bird is updated in its own thread. The function begins by cloning the original flock into an Arc, which allows the birds' data to be safely shared across threads without ownership issues. For each bird in the original flock, a new thread is spawned. Inside each thread, the bird is updated using the update function, which reads from the shared flock (Arc<Vec<Bird>>) to calculate its new state based on the positions and velocities of its neighbours.

Because each thread works on a separate copy of its bird and only reads from shared data, there is no need for synchronization mechanisms like Mutex. This makes the logic simpler, and each thread can proceed without blocking others. Once all threads are spawned, the function collects their results using .join() to retrieve the updated birds and combines them into a final vector, which represents the fully updated flock.

Chunk Thread Lock Free:

This threading strategy parallelizes bird updates by dividing the flock into equally sized chunks and assigning each chunk to a separate thread. Each thread receives a read-only reference to the shared flock (wrapped in Arc) and a copied vector of birds it is

responsible for updating. The birds in each chunk are updated independently in parallel without any need for synchronization primitives like Mutex.

Since each thread receives its own copy of the birds it modifies, there is no shared mutable state and thus no need for locking. The shared flock data (`Arc<Vec<Bird>>`) is used only for reading neighbours' positions and velocities. This makes the model highly efficient and scalable, especially on systems with many cores, as threads can run truly concurrently without blocking each other.

After all threads complete, the updated birds from each chunk are collected and merged back into a single `Vec<Bird>`. The main benefit of this approach is that it avoids contention completely, allowing for smoother parallel execution and better CPU utilization compared to the mutex-based chunked model.

Performance metric:

To measure the performance of the different flocking update models (chunked threads with locks, thread-per-bird, and lock-free chunks), a benchmarking framework called Criterion.rs was used. Criterion automates performance testing by repeatedly executing functions and recording how long they take to run, which allows for consistent and statistically meaningful results. The goal was to compare how well each model handled different flock sizes (500, 1000, 5000, and 10000 birds).

For each model:

- A flock of a specified size was created by generating a vector of randomly initialized birds.
- The corresponding update function (e.g., `update_flock_in_threads`) was repeatedly run inside a measurement loop using Criterion's `b.iter` method. This ensures the code is executed enough times to get a reliable average runtime.
- Before each benchmark iteration, the flock was cloned to avoid modifying the original data, ensuring consistency between tests.
- Each test was registered with Criterion under a unique label (e.g., `chunk_thread_5000`) for clarity in the final report.

During benchmarking, Criterion attempts to gather 100 samples of each function's performance over a default measurement time of 5 seconds. In cases where the flock size was large and execution slower, Criterion warned that it could not collect the full 100 samples within the time limit. This indicates the need to either increase the measurement time or reduce the sample size for accurate metrics on more computationally intensive tasks this will be made clear if this happened in the tables below.

Chunk Thread Locked

Birds Count	Time (min)	Time (mean)	Time (max)	Samples
500	6.5221 ms	6.5488 ms	6.5808 ms	100
1000	12.902 ms	12.960 ms	13.029 ms	100
5000	195.39 ms	196.40 ms	197.66 ms	20 >
10000	743.12 ms	745.42 ms	747.95 ms	10 >

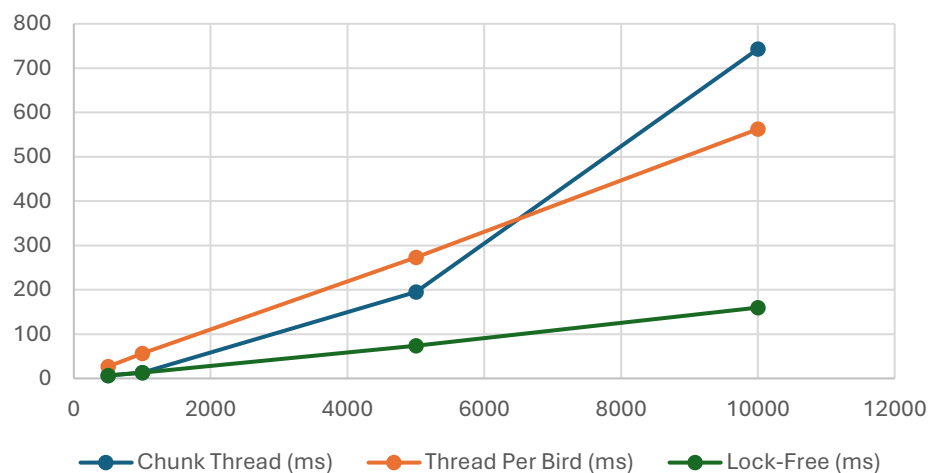
Thread Per Bird

Birds Count	Time (min)	Time (mean)	Time (max)	Samples
500	26.474 ms	26.682 ms	26.897 ms	100
1000	56.440 ms	56.892 ms	57.403 ms	100
5000	272.77 ms	275.22 ms	277.85 ms	10 >
10000	562.51 ms	565.89 ms	569.52 ms	10 >

Chunk Thread Lock Free

Birds Count	Time (min)	Time (mean)	Time (max)	Samples
500	6.5474 ms	6.5846 ms	6.6277 ms	100
1000	12.746 ms	12.813 ms	12.883 ms	100
5000	73.659 ms	74.658 ms	75.735 ms	60 >
10000	159.76 ms	161.57 ms	163.56 ms	30 >

Model Performance



Conclusion:

The benchmarking results clearly demonstrate how each threading model scales with increasing numbers of birds in the flocking simulation. For lower bird counts (500 and 1000), the chunked thread pool model offers similar performance to the chunk thread lock free model, consistently producing the lowest execution times. This indicates that, at smaller scales, the overhead of more complex synchronization mechanisms or excessive thread management is unnecessary and potentially detrimental to performance.

As the number of birds increases to 5000 and 10,000, the lock-free model significantly outperforms both the chunked and thread-per-bird models. While initially comparable in performance to the chunked approach, the lock-free design exhibits superior scalability, maintaining reasonable execution times even at high simulation scales. This suggests that reducing contention and avoiding locking mechanisms becomes increasingly beneficial as workload concurrency grows.

The thread-per-bird model, by contrast, consistently performs the worst across most tested scales. Its overhead in spawning and managing a thread for each bird leads to inefficient resource usage, particularly evident at higher bird counts where execution times become prohibitively high. However it does eventually perform better than the chunk thread locked model just showing how bad the scalability of that model.

In conclusion, the best overall model is the lock free model as it has comparable speeds up the chunk thread locked at lower bird counts but its scalability makes it far and away the best model as it is able to produce far lower values than that of the other two models. The chunk thread locked model could be considered for use cases where the flock sizes would be small, while the thread per bird model should never be used as it is extremely inefficient.

Cuda Lab Book

Week 1 – Lab A

Date: 1 Jan 2025

Exercise 1. Set up CUDA project in Visual Studio 2019 or 2022

Solution:

```
int main()

{   const int arraySize = 5;   const int a[arraySize] = {
    1, 2, 3, 4, 5 }; // Array a   const int b[arraySize] = { 10,
    20, 30, 40, 50 }; // Array b   int c[arraySize] = { 0 }; //
    Array to store the result

    int* dev_a = 0;   int*
    dev_b = 0;   int* dev_c =
    0;   cudaError_t
    cudaStatus;

    cudaStatus = cudaSetDevice(0);   if (cudaStatus != cudaSuccess) {
    fprintf(stderr, "cudaSetDevice failed! Do you have a CUDA-capable GPU
    installed?\n");
    return 1;
    }

    cudaStatus = cudaMalloc((void**)&dev_a, arraySize *
    sizeof(int));   if (cudaStatus != cudaSuccess) {   fprintf(stderr,
    "cudaMalloc failed!\n");   return 1;
    }
}
```

```

    cudaStatus = cudaMalloc((void**)&dev_b, arraySize *
sizeof(int)); if (cudaStatus != cudaSuccess){    fprintf(stderr,
"cudaMalloc failed!\n");    return 1;
}

```

```

    cudaStatus = cudaMalloc((void**)&dev_c, arraySize *
sizeof(int)); if (cudaStatus != cudaSuccess){    fprintf(stderr,
"cudaMalloc failed!\n");    return 1;
}

```

```

    cudaStatus = cudaMemcpy(dev_a, a, arraySize * sizeof(int),
cudaMemcpyHostToDevice); if (cudaStatus != cudaSuccess){    fprintf(stderr,
"cudaMemcpy failed!\n");    return 1;
}

```

```

    cudaStatus = cudaMemcpy(dev_b, b, arraySize * sizeof(int),
cudaMemcpyHostToDevice); if (cudaStatus != cudaSuccess){    fprintf(stderr,
"cudaMemcpy failed!\n");    return 1;
}

```

```

addKernel << <1, arraySize >> > (dev_c, dev_a, dev_b);

```

```

    cudaStatus = cudaGetLastError(); if (cudaStatus != cudaSuccess){
fprintf(stderr, "addKernel launch failed: %s\n", cudaGetErrorString(cudaStatus));
    return 1;
}

```

```

    cudaStatus =
cudaDeviceSynchronize(); if
(cudaStatus != cudaSuccess){
    fprintf(stderr, "cudaDeviceSynchronize returned error code %d after launching
addKernel!\n", cudaStatus);
}

```

```

        return 1;
    }

    cudaStatus = cudaMemcpy(c, dev_c, arraySize * sizeof(int),
cudaMemcpyDeviceToHost); if (cudaStatus != cudaSuccess) {    fprintf(stderr,
"cudaMemcpy failed!\n");    return 1;
}

printf("{1,2,3,4,5} + {10,20,30,40,50} = {%d,%d,%d,%d,%d}\n", c[0], c[1], c[2], c[3], c[4]);

    cudaStatus = cudaDeviceReset(); if
(cudaStatus != cudaSuccess) {
fprintf(stderr, "cudaDeviceReset failed!\n");
return 1;
}

    cudaFree(dev_a);
cudaFree(dev_b);
cudaFree(dev_c);

    return 0;
}

```

Test data:

n/a

Sample output:

```
Microsoft Visual Studio Debug Console
1,2,3,4,5} + {10,20,30,40,50} = {11,22,33,44,55}

C:\Users\725231\source\repos\Lab1\x64\Debug\Lab1.exe (process 7640) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Reflection:

The task was to recreate the add with CUDA method in the main method.

First you need to create two arrays to for the data your adding together and then you need to create 3 pointers to store the arrays in the GPU's memory.

Cuda Malloc allocates memory to store the arrays and then CudaMemcpy is used to copy the data from the CPU to GPU.

Then you use Kernel to add the arrays together. 1 block containing five threads are used because the array length is five.

CUDA synchronise ensures each thread has finished and then the result is copied back to the CPU and outputted before the device is cleaned up and reset.

Further information:

Q2. Exercise 2. CUDA error checking

Solution:

```
int main()

{   const int arraySize = 5;   const int

a[arraySize] = { 1, 2, 3, 4, 5 };   const int

b[arraySize] = { 10, 20, 30, 40, 50 };   int

c[arraySize] = { 0 };

    int* dev_a = 0;   int*

dev_b = 0;   int* dev_c =
```

```
0; cudaError_t
```

```
cudaStatus;
```

```
    cudaStatus = cudaSetDevice(0); if (cudaStatus != cudaSuccess) {  
fprintf(stderr, "cudaSetDevice failed! Do you have a CUDA-capable GPU  
installed?\n");  
    return 1;  
}
```

```
    cudaStatus = cudaMalloc((void**)&dev_a, arraySize * sizeof(int)); if (cudaStatus !=  
cudaSuccess) {    fprintf(stderr, "cudaMalloc for dev_a failed! Error: %s\n",  
cudaGetErrorString(cudaStatus));    goto Error;  
}
```

```
    cudaStatus = cudaMalloc((void**)&dev_b, arraySize * sizeof(int)); if (cudaStatus !=  
cudaSuccess) {    fprintf(stderr, "cudaMalloc for dev_b failed! Error: %s\n",  
cudaGetErrorString(cudaStatus));    goto Error;  
}
```

```
    cudaStatus = cudaMalloc((void**)&dev_c, arraySize * sizeof(int)); if (cudaStatus !=  
cudaSuccess) {    fprintf(stderr, "cudaMalloc for dev_c failed! Error: %s\n",  
cudaGetErrorString(cudaStatus));    goto Error;  
}
```

```
    cudaStatus = cudaMemcpy(dev_a, a, arraySize * sizeof(int), cudaMemcpyHostToDevice);  
    if (cudaStatus != cudaSuccess) {    fprintf(stderr, "cudaMemcpy for dev_a failed! Error:  
%s\n", cudaGetErrorString(cudaStatus));  
        goto Error;  
}
```

```

    cudaStatus = cudaMemcpy(dev_b, b, arraySize * sizeof(int), cudaMemcpyHostToDevice);
if (cudaStatus != cudaSuccess) {    fprintf(stderr, "cudaMemcpy for dev_b failed! Error:
%s\n", cudaGetErrorString(cudaStatus));
    goto Error;
}

addKernel << <1, arraySize >> > (dev_c, dev_a, dev_b);

    cudaStatus = cudaGetLastError();  if (cudaStatus != cudaSuccess) {
fprintf(stderr, "addKernel launch failed! Error: %s\n", cudaGetErrorString(cudaStatus));
goto Error;
}

    cudaStatus =
cudaDeviceSynchronize();  if
(cudaStatus != cudaSuccess) {
    fprintf(stderr, "cudaDeviceSynchronize returned error code %d after launching
addKernel!\n", cudaStatus);
    goto Error;
}

    cudaStatus = cudaMemcpy(c, dev_c, arraySize * sizeof(int), cudaMemcpyDeviceToHost);
if (cudaStatus != cudaSuccess) {    fprintf(stderr, "cudaMemcpy for dev_c failed! Error:
%s\n", cudaGetErrorString(cudaStatus));    goto Error;
}

    printf("{1,2,3,4,5} + {10,20,30,40,50} = {%d,%d,%d,%d,%d}\n", c[0], c[1], c[2], c[3], c[4]);

    cudaStatus = cudaDeviceReset();  if (cudaStatus != cudaSuccess) {
fprintf(stderr, "cudaDeviceReset failed! Error: %s\n", cudaGetErrorString(cudaStatus));
    return 1;
}

```

```

    cudaFree(dev_a);
cudaFree(dev_b);
cudaFree(dev_c);

    return 0;

```

Error:

```

    cudaFree(dev_a);
cudaFree(dev_b);
cudaFree(dev_c);

    return 1;
}

```

Output: Same

as above

Reflection:

You can add checks to ensure that CUDA ran properly. This is probably best practice and good for debugging.

Q3. Exercise 2. Check time range

Solution: int main()

```

{   const int arraySize = 500; // Change to different sizes for testing (e.g., 500, 5000,
etc.)   const int blockSize = 256; // Number of threads per block (can vary)

```

```

    const int numBlocks = (arraySize + blockSize - 1) / blockSize; // Number of blocks, calculated
based on array size

```

```

    int* a = new int[arraySize]; // Dynamically allocate arrays for large
size   int* b = new int[arraySize];   int* c = new int[arraySize];

```

```

    // Initialize arrays a and b
for (int i = 0; i < arraySize; ++i) {
a[i] = i + 1;    b[i] = (i + 1) * 10;

```

```

}

int* dev_a = 0; int*
dev_b = 0; int* dev_c =
0; cudaError_t
cudaStatus;

// Step 1: Set the device to use GPU 0  cudaStatus = cudaSetDevice(0);  if
(cudaStatus != cudaSuccess) {    fprintf(stderr, "cudaSetDevice failed! Do you have a
CUDA-capable GPU installed?\n");

    return 1;
}

// Step 2: Allocate memory on the device (GPU)  cudaStatus =
cudaMalloc((void**)&dev_a, arraySize * sizeof(int));  if (cudaStatus != cudaSuccess) {
fprintf(stderr, "cudaMalloc for dev_a failed! Error: %s\n", cudaGetErrorString(cudaStatus));

    return 1;
}

    cudaStatus = cudaMalloc((void**)&dev_b, arraySize * sizeof(int));  if (cudaStatus !=
cudaSuccess) {    fprintf(stderr, "cudaMalloc for dev_b failed! Error: %s\n",
cudaGetErrorString(cudaStatus));

    return 1;
}

    cudaStatus = cudaMalloc((void**)&dev_c, arraySize * sizeof(int));  if (cudaStatus !=
cudaSuccess) {    fprintf(stderr, "cudaMalloc for dev_c failed! Error: %s\n",
cudaGetErrorString(cudaStatus));

    return 1;
}

```

```
    // Step 3: Copy input data (arrays a and b) from host (CPU) to device (GPU)    cudaStatus =  
    cudaMemcpy(dev_a, a, arraySize * sizeof(int), cudaMemcpyHostToDevice);    if (cudaStatus  
    != cudaSuccess) {        fprintf(stderr, "cudaMemcpy for dev_a failed! Error: %s\n",  
    cudaGetErrorString(cudaStatus));  
        return 1;  
    }
```

```
    cudaStatus = cudaMemcpy(dev_b, b, arraySize * sizeof(int), cudaMemcpyHostToDevice);  
    if (cudaStatus != cudaSuccess) {        fprintf(stderr, "cudaMemcpy for dev_b failed! Error:  
    %s\n", cudaGetErrorString(cudaStatus));  
        return 1;  
    }
```

```
    // Step 4: Create CUDA events for  
    timing    cudaEvent_t start, stop;    float  
    elapsedTime;  
    cudaEventCreate(&start);  
    cudaEventCreate(&stop);
```

```
    // Record the start time  
    cudaEventRecord(start, 0);
```

```
    // Step 5: Launch the kernel    addKernel << <numBlocks,  
    blockSize >>> (dev_c, dev_a, dev_b);
```

```
    // Check for kernel launch errors    cudaStatus = cudaGetLastError();    if (cudaStatus  
    != cudaSuccess) {        fprintf(stderr, "addKernel launch failed! Error: %s\n",  
    cudaGetErrorString(cudaStatus));  
        return 1;  
    }
```

```
// Step 6: Synchronize and record the end time  cudaEventRecord(stop, 0);  
cudaEventSynchronize(stop); // Ensure the event is completed before measuring  
time
```

```
// Calculate the time elapsed  
cudaEventElapsedTime(&elapsedTime, start, stop);  printf("Time  
elapsed for kernel execution: %f ms\n", elapsedTime);
```

```
// Step 7: Copy the result back from device (GPU) to host (CPU)  cudaStatus =  
cudaMemcpy(c, dev_c, arraySize * sizeof(int), cudaMemcpyDeviceToHost);  if (cudaStatus  
!= cudaSuccess) {      fprintf(stderr, "cudaMemcpy for dev_c failed! Error: %s\n",  
cudaGetErrorString(cudaStatus));  
    return 1;  
}
```

```
// Output the result  
// printf("{1,2,3,4,5} + {10,20,30,40,50} = {%d,%d,%d,%d,%d}\n", c[0], c[1], c[2], c[3], c[4]);
```

```
// Step 8:  
Cleanup  
cudaFree(dev_a);  
cudaFree(dev_b);  
cudaFree(dev_c);  
delete[] a;  
delete[] b;  
delete[] c;
```

```
// Destroy the events  
cudaEventDestroy(start);  
cudaEventDestroy(stop);
```

```

    return 0;
}

```

Test Data:

256 threads per block.

For Array size 500: Elapsed time = 0.881664 ms

For Array size 5000: Elapsed time = 0.804864 ms

For Array size 50000: Elapsed time = 0.561856 ms

For Array size 50000000: Elapsed time = 2.488448

ms

Reflection:

You can clearly see that the GPU time does not change much as it can handle the data simultaneously and it has enough threads to use. Until you get to very high array sizes at which point there needs to be more blocks made which can increase overhead. If I increased the threads per block time would go down for larger arrays. You get more use out of parallel calculations the larger the workload up to a point. If its smaller you get diminishing returns.

Week 2 – Lab B

Exercise 1. Understand the block and thread indices

1) addKernel << <1, 5 >> > (dev_c, dev_a, dev_b);

Thread	Thread.Id.x	Block.Id.x
1	0	0
2	1	0
3	2	0
4	3	0
5	4	0

2) addKernel << <2, 3 >> > (dev_c, dev_a, dev_b);

Thread	Thread.Id.x	Block.Id.x
1	0	0
2	1	0
3	2	0
4	0	1
5	1	1
6	2	1

3) addKernel << <3, 2 >> > (dev_c, dev_a, dev_b);

Thread	Thread.Id.x	Block.Id.x
1	0	0
2	1	0
3	0	1
4	1	1
5	0	2
6	1	2

4) addKernel << <6, 1 >> > (dev_c, dev_a, dev_b);

Thread	Thread.Id.x	Block.Id.x
1	0	0
2	0	1
3	0	2
4	0	3
5	0	4
6	0	5

Reflection: I now understand how threads and blocks are structured. Threads are inside blocks and are given id's based on how they are initialised in the block. Block Ids' are assigned based on when they were initialised in the kernel.

Exercise 2. Find vector addition using multiple 1D thread blocks

Solution: $i = \text{threadIdx.x} + \text{blockIdx.x} * \text{blockDim.x}$;

Test data:

Block 0, Thread 0: $i = 0 + 0 * 2 = 0$

Block 0, Thread 1: $i = 1 + 0 * 2 = 1$

Block 1, Thread 0: $i = 0 + 1 * 2 = 2$

Block 1, Thread 1: $i = 1 + 1 * 2 = 3$

Block 2, Thread 0: $i = 0 + 2 * 2 = 4$

Block 2, Thread 1: $i = 1 + 2 * 2 = 5$

Reflection: When using multiple blocks and threads in CUDA you need to properly manage the thread indices to ensure that each thread works on its own unique data. Without calculating a global thread index, threads within different blocks could end up referencing the same array elements. By calculating the global index (with $\text{threadIdx.x} + \text{blockIdx.x} * \text{blockDim.x}$), you ensure that each thread accesses a unique element of the arrays, avoiding overlap or missed elements.

Exercise 3. Understand the thread indices for 2D blocks

1) addKernel << <1, dim3(2, 3) >> > (dev_c, dev_a, dev_b);

threadIdx.x	threadIdx.y
0	0
1	0
0	1
1	1
0	2
1	2

2) addKernel << <1, dim3(3, 3) >> > (dev_c, dev_a, dev_b);

threadIdx.x	threadIdx.y
0	0
1	0
2	0
0	1
1	1
2	1
0	2
1	2
2	2

Reflection: This exercise helped me understand how thread IDs are organized in a 2D thread block in CUDA. The thread IDs work similarly to elements in a 2D array, with each thread having a unique combination of indices (threadIdx.x and threadIdx.y) that allows it to access specific data. The overall structure of the threads forms a 2D grid, where each thread is assigned, a specific position based on its x and y coordinates within the block.

Exercise 4. Find vector addition using one 2D thread block

Solution: `int i = threadIdx.x + blockDim.x * threadIdx.y;`

Test data:

Thread.Idx.x	Thread.Idx.y	Global index
0	0	0
1	0	1
2	0	2
0	1	3
1	1	4

2	1	5
---	---	---

Reflection: I learned how thread indices are used in a 2D block configuration to calculate unique global indices for each thread. By utilizing the threadIdx.x and threadIdx.y values, each thread in the block can be assigned a specific index that corresponds to an element in the data arrays.

Exercise 5. Find vector addition using multiple 2D thread blocks

Solution: $\text{int } i = \text{threadIdx.x} + \text{blockDim.x} * \text{threadIdx.y} + \text{blockIdx.x} * \text{blockDim.x} * \text{blockDim.y} + \text{blockIdx.y} * \text{blockDim.x} * \text{blockDim.y} * \text{gridDim.x};$

Test data:

Block (0, 0):

Thread (0, 0): $i = 0 + 2 * 0 + 0 * 2 * 2 + 0 * 2 * 2 * 2 = 0$

Thread (1, 0): $i = 1 + 2 * 0 + 0 * 2 * 2 + 0 * 2 * 2 * 2 = 1$

Thread (0, 1): $i = 0 + 2 * 1 + 0 * 2 * 2 + 0 * 2 * 2 * 2 = 2$

Thread (1, 1): $i = 1 + 2 * 1 + 0 * 2 * 2 + 0 * 2 * 2 * 2 = 3$

Block (1, 0):

Thread (0, 0): $i = 0 + 2 * 0 + 1 * 2 * 2 + 0 * 2 * 2 * 2 = 4$

Thread (1, 0): $i = 1 + 2 * 0 + 1 * 2 * 2 + 0 * 2 * 2 * 2 = 5$

Thread (0, 1): $i = 0 + 2 * 1 + 1 * 2 * 2 + 0 * 2 * 2 * 2 = 6$

Thread (1, 1): $i = 1 + 2 * 1 + 1 * 2 * 2 + 0 * 2 * 2 * 2 = 7$

Block (0, 1):

Thread (0, 0): $i = 0 + 2 * 0 + 0 * 2 * 2 + 1 * 2 * 2 * 2 = 8$

Thread (1, 0): $i = 1 + 2 * 0 + 0 * 2 * 2 + 1 * 2 * 2 * 2 = 9$

Thread (0, 1): $i = 0 + 2 * 1 + 0 * 2 * 2 + 1 * 2 * 2 * 2 = 10$

Thread (1, 1): $i = 1 + 2 * 1 + 0 * 2 * 2 + 1 * 2 * 2 * 2 = 11$

Block (1, 1):

Thread (0, 0): $i = 0 + 2 * 0 + 1 * 2 * 2 + 1 * 2 * 2 * 2 = 12$

Thread (1, 0): $i = 1 + 2 * 0 + 1 * 2 * 2 + 1 * 2 * 2 * 2 = 13$

Thread (0, 1): $i = 0 + 2 * 1 + 1 * 2 * 2 + 1 * 2 * 2 * 2 = 14$

Thread (1, 1): $i = 1 + 2 * 1 + 1 * 2 * 2 + 1 * 2 * 2 * 2 = 15$

Block (0, 2):

Thread (0, 0): $i = 0 + 2 * 0 + 0 * 2 * 2 + 2 * 2 * 2 * 2 = 16$

Thread (1, 0): $i = 1 + 2 * 0 + 0 * 2 * 2 + 2 * 2 * 2 * 2 = 17$

Thread (0, 1): $i = 0 + 2 * 1 + 0 * 2 * 2 + 2 * 2 * 2 * 2 = 18$

Thread (1, 1): $i = 1 + 2 * 1 + 0 * 2 * 2 + 2 * 2 * 2 * 2 = 19$

Block (1, 2):

Thread (0, 0): $i = 0 + 2 * 0 + 1 * 2 * 2 + 2 * 2 * 2 * 2 = 20$

Thread (1, 0): $i = 1 + 2 * 0 + 1 * 2 * 2 + 2 * 2 * 2 * 2 = 21$

Thread (0, 1): $i = 0 + 2 * 1 + 1 * 2 * 2 + 2 * 2 * 2 * 2 = 22$

Thread (1, 1): $i = 1 + 2 * 1 + 1 * 2 * 2 + 2 * 2 * 2 * 2 = 23$

Reflection:

Thread Index Calculation within the Block:

$\text{threadIdx.x} + \text{blockDim.x} * \text{threadIdx.y}$: This computes the index for each thread within the current block. It works similarly to how the global index is calculated for 2D blocks in the previous exercises.

Block Index Calculation:

$\text{blockIdx.x} * \text{blockDim.x} * \text{blockDim.y}$: This calculates the offset for each block in the x-direction based on its position in the grid and the total number of threads in each block.

$\text{blockIdx.y} * \text{blockDim.x} * \text{blockDim.y} * \text{gridDim.x}$: This calculates the offset for each block in the y-direction, considering the total number of blocks in the x-direction (gridDim.x).

This task is about how to calculate the global ID in 2D with multiple blocks and threads.

LAB 3 CUDA

Exercise 1. Vector dot-product:

Solution 1: CPU only solution

```
double dotProduct(const std::vector<double>& vec1, const
std::vector<double>& vec2) {    double result = 0.0;    size_t n = vec1.size();

    for (size_t i = 0; i < n; ++i) {        result += vec1[i]
* vec2[i];
    }

    return result;
}

int main() {

    size_t N = 50;

    std::vector<double> vec1(N, 3.0);    std::vector<double> vec2(N, 2.0);

    auto start = std::chrono::high_resolution_clock::now();

    double result = dotProduct(vec1, vec2);
    auto end = std::chrono::high_resolution_clock::now();

    std::chrono::duration<double> duration = end - start;

    std::cout << "Dot Product: " << result << std::endl;
```

```

std::cout << "Time taken: " << duration.count() << " seconds" << std::endl;

return 0;
}

Solution 2: CPU + GPU solution

#include "cuda_runtime.h"

#include "device_launch_parameters.h"

#include <stdio.h>

#include <iostream>

#include <vector>

#include <chrono> // For timing the operation

#define N 1000000 // Size of the vectors

// Kernel to compute element-wise multiplication of two vectors
__global__ void elementWiseMultiplication(const double* a, const double* b, double* c, int
size) {

    int index = threadIdx.x + blockIdx.x * blockDim.x; if (index < size) {

        c[index] = a[index] * b[index];

    }

}

// Function to calculate the dot product by summing up the elements on the CPU
double sumVector(const double* c, int size){ double
result = 0.0; for (int i = 0; i < size; ++i){ result += c[i];

    } return result;

}

int main() {

    // Allocate memory for vectors on host

```

```
std::vector<double> a(N, 20.0); // Vector a filled with 1.0  std::vector<double> b(N, 10.0); //
Vector b filled with 2.0  std::vector<double> c(N, 0.0); // Vector c to store results of
multiplication
```

```
// Allocate memory for vectors on device (GPU)  double*
d_a, * d_b, * d_c;  cudaMalloc(&d_a, N * sizeof(double));
cudaMalloc(&d_b, N * sizeof(double));  cudaMalloc(&d_c, N *
sizeof(double));
```

```
// Copy input vectors from host to device
cudaMemcpy(d_a, a.data(), N * sizeof(double), cudaMemcpyHostToDevice);
cudaMemcpy(d_b, b.data(), N * sizeof(double), cudaMemcpyHostToDevice);
// CUDA events for measuring GPU time  cudaEvent_t startEvent,
stopEvent;  cudaEventCreate(&startEvent);
cudaEventCreate(&stopEvent);
```

```
// Start measuring GPU time  cudaEventRecord(startEvent);
```

```
// Launch kernel for element-wise multiplication  int
threadsPerBlock = 256;
int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;  elementWiseMultiplication
<< <blocksPerGrid, threadsPerBlock >> > (d_a, d_b, d_c, N);
```

```
// Check for kernel launch errors  cudaError_t err
= cudaGetLastError();  if (err != cudaSuccess) {
    std::cerr << "CUDA kernel launch failed: " << cudaGetErrorString(err) << std::endl;
    return -1;  }
```

```
// Copy the result vector c back to the host
cudaMemcpy(c.data(), d_c, N * sizeof(double), cudaMemcpyDeviceToHost);
```

```

    // Stop measuring GPU time    cudaEventRecord(stopEvent);
cudaEventSynchronize(stopEvent); // Calculate elapsed GPU time    float
gpuTime;

    cudaEventElapsedTime(&gpuTime, startEvent, stopEvent);

    // Calculate the sum of vector c on CPU to get the dot product    auto startCpu =
std::chrono::high_resolution_clock::now();    double dotProduct =
sumVector(c.data(), N);    auto endCpu =
std::chrono::high_resolution_clock::now();    std::chrono::duration<double>
cpuTime = endCpu - startCpu;

    // Output the result and time taken

    std::cout << "Dot Product: " << dotProduct << std::endl;    std::cout << "GPU Time: " <<
gpuTime << " milliseconds" << std::endl;

    std::cout << "CPU Time for summing: " << cpuTime.count() << " seconds" << std::endl;

    // Clean up device memory
cudaFree(d_a);    cudaFree(d_b);
cudaFree(d_c);

    // Clean up CUDA events    cudaEventDestroy(startEvent);
cudaEventDestroy(stopEvent);

    return 0;
}

```

Reflection: When testing different sized vectors, I observed that the CPU times were lower than the GPU times for smaller vectors. This is likely due to the overhead associated with launching a GPU kernel. For smaller vectors, the time taken to transfer data between the host (CPU) and device (GPU), as well as the kernel launch overhead, outweighs the parallel processing benefits of using the GPU. The CPU, on the other hand, can process smaller vectors quickly due to its low-latency operations and lack of such overhead.

As the size of the vectors increases, the GPU becomes more efficient because it can process larger datasets in parallel, leading to significant speedups. The increased parallelism helps the

GPU handle large vectors much faster than the CPU, making it a better choice for larger workloads.

Task 2: Vector dot-product using unified memory.

Solution:

```
__device__ __managed__ int a[5], b[5], c[5]; // Declaring managed arrays

__global__ void PerElement_AtimesB(int* c, int* a, int* b) {  int index =
threadIdx.x;

    if (index < 5) { // Ensure thread does not go out of bounds      c[index] = a[index] *
b[index]; // Element-wise multiplication  }
}

int main() {

    // Initialize vectors a and b on the host  for (int i = 0; i
< 5; i++) {
        a[i] = i + 1;    // a = {1, 2, 3, 4, 5}    b[i] = (i + 1) * 10; // b =
{10, 20, 30, 40, 50}  }

    // Launch kernel to compute element-wise multiplication
    PerElement_AtimesB <<<1, 5>>> (c, a, b);

    // Wait for GPU kernel to finish execution  cudaDeviceSynchronize();

    // Print the result of element-wise multiplication
    printf("{1,2,3,4,5} + {10,20,30,40,50} = {%d,%d,%d,%d,%d}\n", c[0], c[1], c[2], c[3], c[4]);

    // Now, calculate the dot-product of vectors a and b  int
dotProduct = 0;  for (int i = 0; i < 5; i++) {
        dotProduct += c[i]; // Summing the elements of c to get the dotproduct
    }
```

```

// Print the dot product

std::cout << "Dot Product: " << dotProduct << std::endl;

return 0;
}

```

Reflection:

The vectors a, b, and c are declared using the `__device__ __managed__` keywords. This ensures that these arrays are allocated in memory that is accessible both from the host (CPU) and the device (GPU). This simplifies memory management because there's no need to use `cudaMallocManaged()` or `cudaMemcpy()`.

The use of `__device__ __managed__` simplifies the code by allowing the same memory to be accessible from both the host and device. This eliminates the need for explicit memory transfers between the host and device, which reduces the complexity and size of the code making it far more readable.

Exercise 3. Vector dot-product using shared Memory

Solution:

```

__global__ void PerElement_At看imesB(int *c, int *a, int *b) {
    __shared__ int
dataPerBlock[4]; // Shared memory for 4 elements

    int i = blockIdx.x * blockDim.x + threadIdx.x; // Global index of element
dataPerBlock[threadIdx.x] = a[i] * b[i]; // Load data into shared
memory

    __syncthreads(); // Synchronize threads in the block before proceeding

    // Perform reduction to calculate subtotal for this block
    float subtotal = 0;
    for (int k = 0; k < blockDim.x; k++) {
        subtotal +=
dataPerBlock[k];
    }

    __syncthreads(); // Synchronize to ensure subtotal is fully calculated before writing

```

```

    // Write subtotal to global memory (one per block)  c[blockIdx.x] = subtotal;
}

```

Reflection: I learned how to implement thread synchronization using `__syncthreads()` to manage shared memory correctly in CUDA. This was essential to prevent race conditions and ensure that all threads finished loading data and computing results before proceeding to the next step.

Part 2 of the exercise:

Solution: The code cannot work for some of the thread configurations.

<<<1, 8>>>: Requires shared memory size to be 8 to accommodate all 8 elements.

<<<2, 4>>>: Works as expected with no modification needed.

<<<4, 2>>>: Requires adjusting the shared memory size to 2 and possibly modifying the kernel logic to ensure proper indexing and memory usage.

```

__global__ void PerElement_AtimesB(int *c, int *a, int *b) {  int i = blockIdx.x *
blockDim.x + threadIdx.x;

```

```

    // Modify the shared memory size according to the number of threads in a block

```

```

    __shared__ int dataPerBlock[8]; // Use 8 for <<<1, 8>>> configuration or adjust dynamically

```

```

    // Load data into shared memory

```

```

    if (i < 8) {
        dataPerBlock[threadIdx.x] = a[i] * b[i];  }

```

```

    __syncthreads(); // Synchronize threads to make sure shared memory is loaded

```

```

    // Compute subtotal for this block  float subtotal

```

```

    = 0;
    for (int k = 0; k < blockDim.x; k++) {        subtotal +=
dataPerBlock[k];
    }

```

```

    __syncthreads(); // Synchronize threads before writing the subtotal

```

```
// Write subtotal to global memory  
c[blockIdx.x] = subtotal; } adjusted kernel  
code.
```

Reflection:

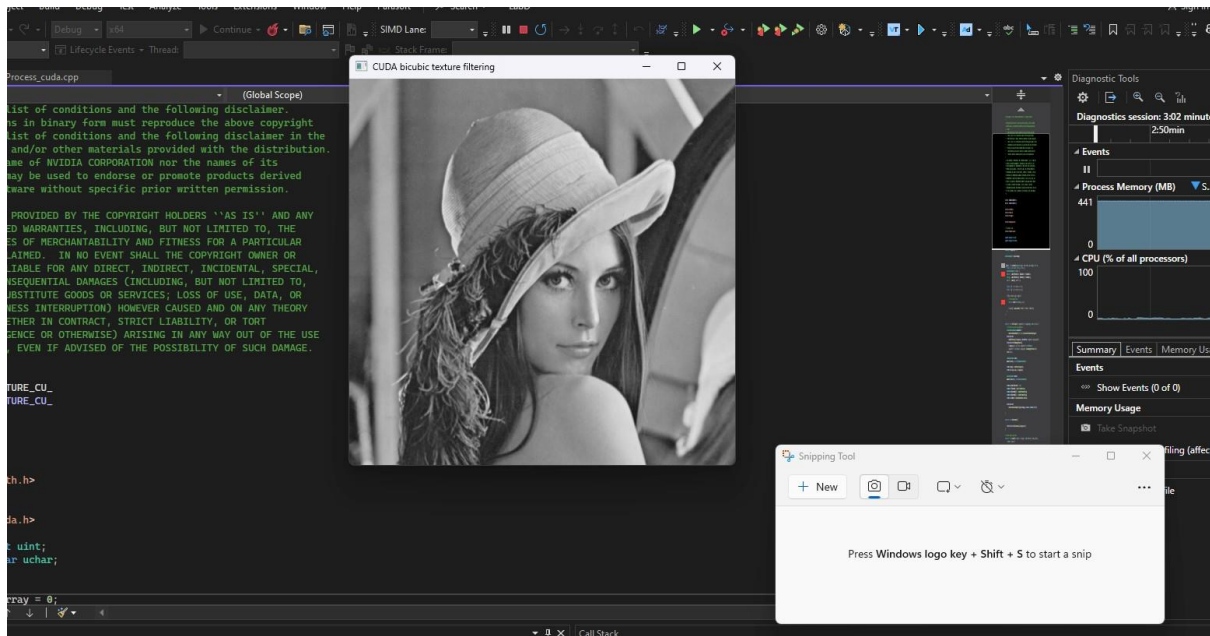
I explored the impact of different thread configurations on the performance and correctness of the vector dot-product program. I realized that using mismatched thread and block sizes could lead to issues with indexing and memory allocation, and learned how to adjust shared memory sizes and indexing logic to make the program work correctly across various configurations.

The global index i adjusts automatically to accommodate the varying number of blocks and threads, allowing each thread to process the correct data element, even with different configurations like $\langle\langle\langle 1, 8 \rangle\rangle\rangle$, $\langle\langle\langle 2, 4 \rangle\rangle\rangle$, or $\langle\langle\langle 4, 2 \rangle\rangle\rangle$.

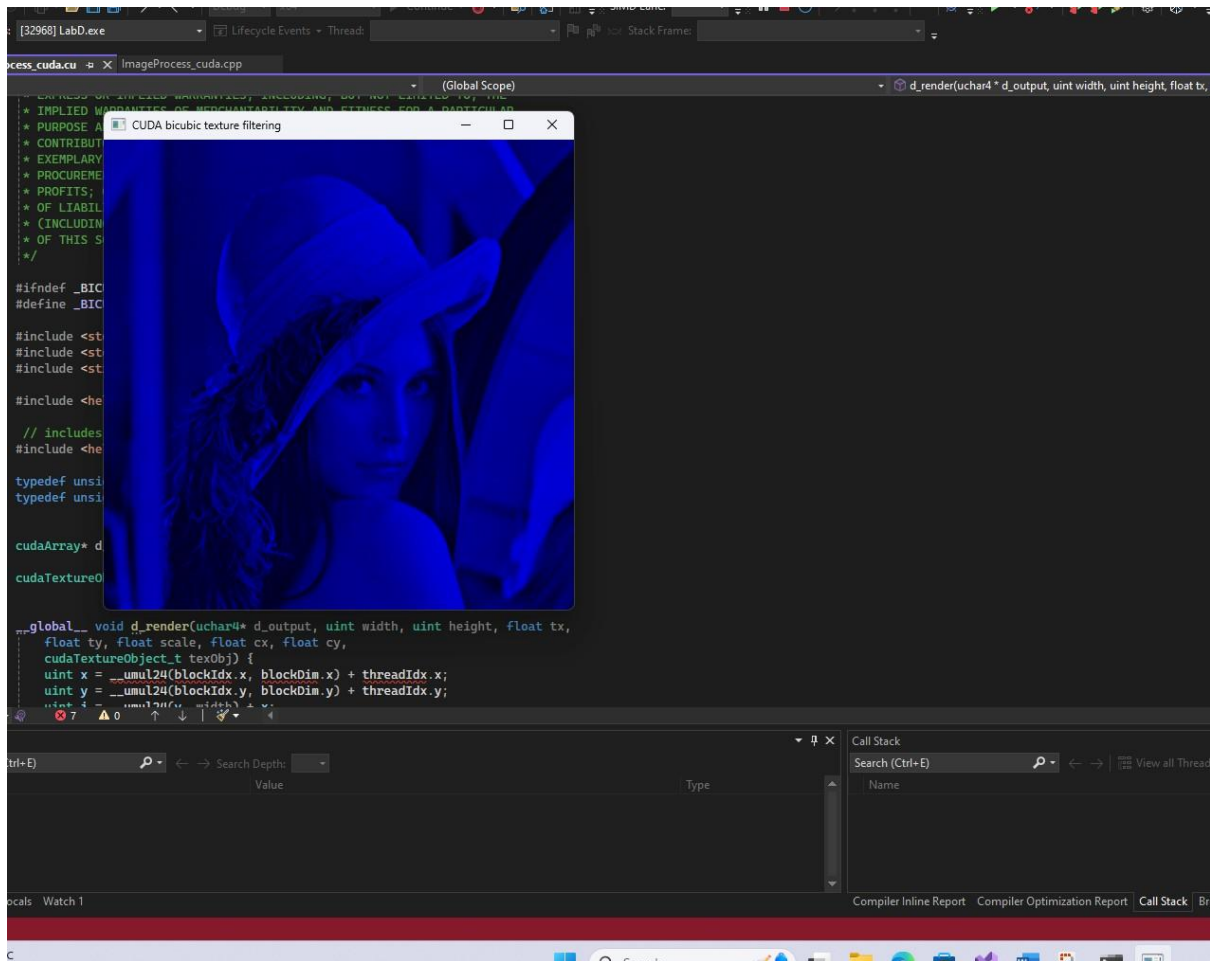
Shared memory is quicker than managed memory as it allows threads in the same block to access data quicker than managed memory. By allocating shared memory inside the kernel intermediate results like subtotals are stored within each thread block, which drastically reduced memory access times compared to global memory.

Lab 4 CUDA

Exercise 1:

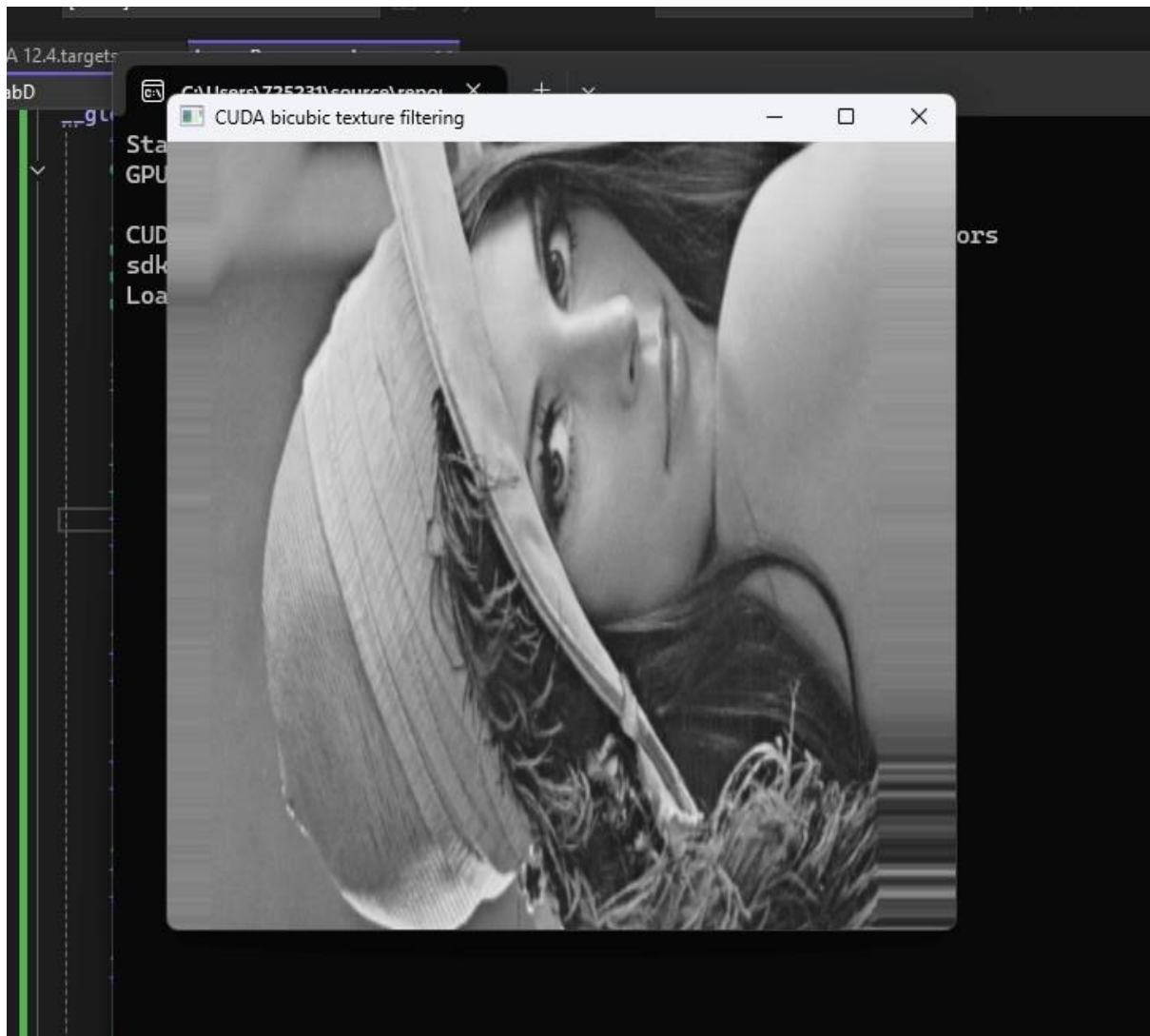


Reflection: I have changed the directories and added libraries successfully to allow image generation in Cuda.



Reflection/Solution `d_output[i] = make_uchar4(c * 255, 0, 0, 255);`

By changing this line in the render class we can change the colour of the image. In this case we have changed the colour to blue. The four values represented are RGB and alpha. Apparently some libraries have different orderings which is BGRA which is probably why my image was blue instead of red.



```
// Define transformation parameters float2 T = { 20, 10 }; // Translation vector
(move right by 20, down by 10) float2 S = { 1.2, 0.5 }; // Scaling factors (1.2x
width, 0.5x height) float angle = 1.5708; // Rotation angle in radians float
centerX = width / 2.0f; // Center for transformations float centerY = height / 2.0f;
```

```
// Translate float
tx_x = x + T.x; float
ty_y = y + T.y; //
Scale around center
(cx, cy) float sx =
(tx_x - centerX) * S.x
+ centerX; float sy =
(ty_y - centerY) * S.y
+ centerY;
```

```
// Rotate around center (cx, cy) float rx = (sx - centerX) * cos(angle) - (sy - centerY) *  
sin(angle) + centerX; float ry = (sx - centerX) * sin(angle) + (sy - centerY) * cos(angle)  
+ centerY;
```

Reflection:

This exercise provided insight into image processing in CUDA, where each pixel is processed in parallel using a 2D grid of threads. I learned how to manipulate pixel coordinates to apply transformations like translation, scaling, and rotation. Using texture memory for efficient image sampling, I implemented a 90-degree rotation by adjusting pixel positions mathematically.

Lab 5 Cuda

Exercise 1: CPU `#include <iostream>`

```
const int heightA = 4; const int
widthA = 3; const int heightB = 3;
const int widthB = 2;

// C = A x B
void matrixMultiplyHost(const int *A, const int *B, int *C) { for (int i = 0; i <
heightA; i++) { for (int j = 0; j < widthB; j++) { int sum = 0;
    for (int k = 0; k < widthA; k++) { sum += A[i * widthA + k] *
B[k * widthB + j];
    }
    C[i * widthB + j] = sum;
  }
}

int main() {
    int A[heightA * widthA] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12}; int B[heightB *
widthB] = {1, 2, 3, 4, 5, 6}; int C[heightA * widthB] = {0};

    matrixMultiplyHost(A, B, C);

    std::cout << "Resultant Matrix C:\n"; for (int i = 0; i <
heightA; i++) { for (int j = 0; j < widthB; j++) {
std::cout << C[i * widthB + j] << " ";
    }
    std::cout << "\n";
```

```

    }

    return 0;
}

```

Output:

Resultant Matrix C (CUDA):

```

22 28
49 64
76 100
103 136

```

Reflection:

Implementing matrix multiplication on the CPU helped solidify my understanding of how the dot product is computed for each element in the result matrix. I learned that for two matrices A (of size $m \times n$) and B (of size $n \times p$), each element $C[i][j]$ in the resulting matrix C (of size $m \times p$) is computed as:

$$C[i][j] = \sum_{k=0}^{n-1} A[i][k] \times B[k][j]$$

This formula highlights that each output element is the sum of products from a row in A and a column in B.

Since arrays in C++ are stored in row-major order, I needed to correctly compute the indices for accessing matrix elements using:

$$\text{index} = \text{row} \times \text{width} + \text{col}$$

This helped me understand how to navigate a flattened 1D representation of a 2D matrix.

Exercise 2: CUDA Solution To Matrix Multiplication

```

#include <iostream> #include
<cuda_runtime.h>

const int heightA = 4; const int
widthA = 3; const int heightB = 3;
const int widthB = 2;

```

```
const int arraySizeA = heightA * widthA; const int  
arraySizeB = heightB * widthB; const int arraySizeC =  
heightA * widthB;
```

```
// CUDA Kernel
```

```
__global__ void matrixMultiplyCUDA(int* A, int* B, int* C, int widthA, int widthB) {  
    int row = threadIdx.y;    int col =  
    threadIdx.x;
```

```
  
    if (row < heightA && col < widthB) {        int sum = 0;  
        for (int k = 0; k < widthA; k++) {  
            sum += A[row * widthA + k] * B[k * widthB + col];  
        }  
        C[row * widthB + col] = sum;  
    }  
}
```

```
int main() {  
    int h_A[arraySizeA] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 };    int h_B[arraySizeB]  
= { 1, 2, 3, 4, 5, 6 };    int h_C[arraySizeC] = { 0 };  
  
    int* d_A, * d_B, * d_C;
```

```
///cuda stuff was here
```

```
  
    // Print the result  
    std::cout << "Resultant Matrix C (CUDA):\n";    for (int i = 0;  
i < heightA; i++) {        for (int j = 0; j < widthB; j++) {  
std::cout << h_C[i * widthB + j] << " ";  
        }  
        std::cout << "\n";
```

```
}  
}
```

Output:

Resultant Matrix C (CUDA):

22 28

49 64

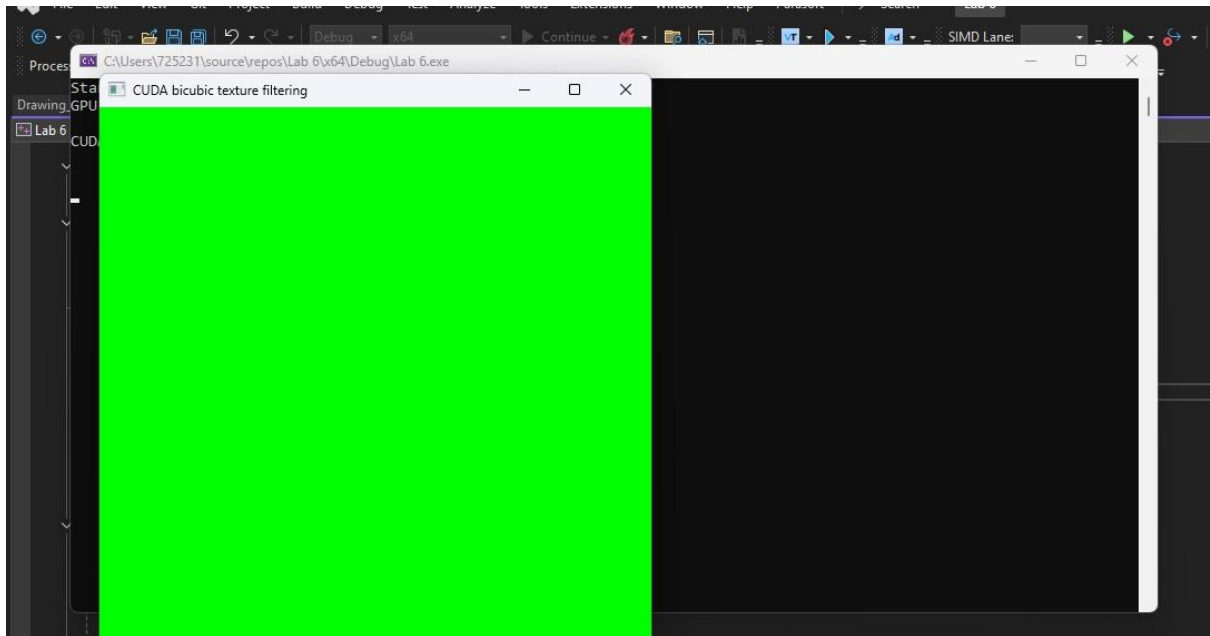
76 100

103 136

Reflection: In this task I remembered how matrix multiplication works and learned how to make matrix multiplication work in cuda. Matrix multiplication is done by taking the dot product from the corresponding rows and columns. In this task I flattened the matrixes into 1D arrays then used row major order to get the correct element. $\text{index} = \text{row} \times \text{widthA} + \text{col}$. threadIdx.x corresponds to column index in matrix C threadIdx.y corresponds to row index in matrix C. This allows each thread to compute one element of C.

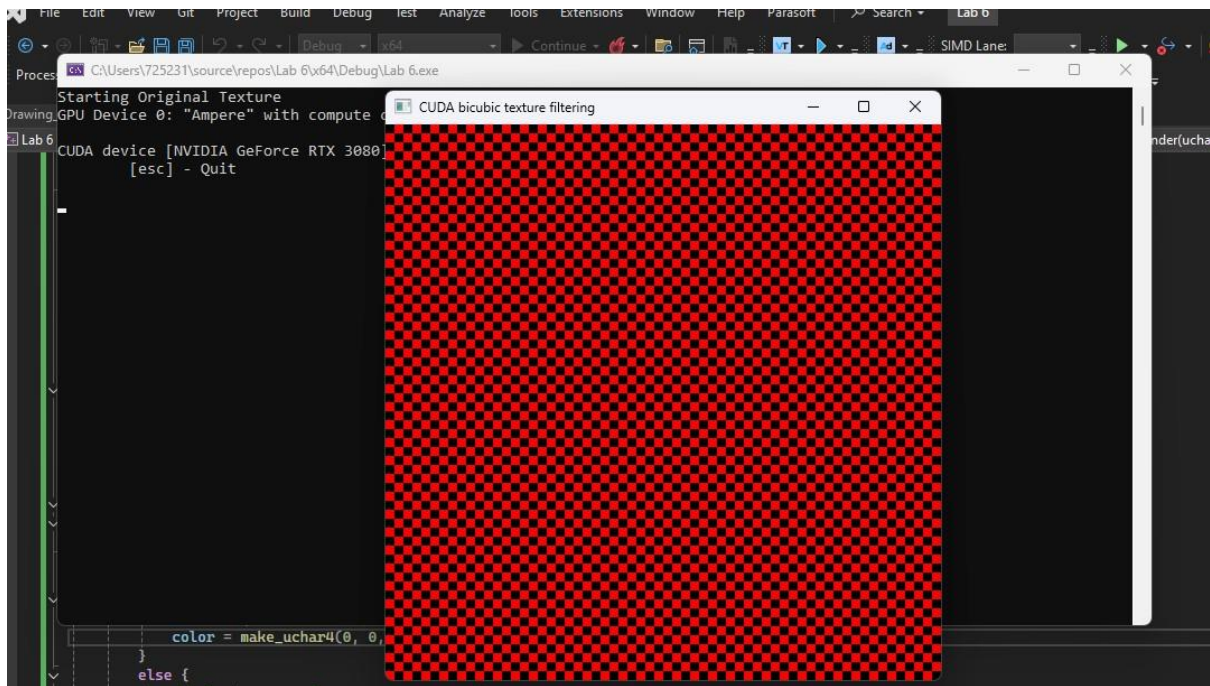
Lab 6 CUDA

Exercise 1: Set up a virtual canvas and draw on it an image in CUDA



Reflection: I used the repository from lab 4 adding the directories and libraries to make the code work. I then changed the colour in the render method by changing the values `d_output[i] = make_uchar4(0, 0xff, 0, 0);` Mine is different from the lab it is BGR rather than RGB.

Exercise 2: Drawing a checkboard



```
__global__ void d_render(uchar4* d_output, uint width, uint height) { uint x =
__umul24(blockIdx.x, blockDim.x) + threadIdx.x; uint y = __umul24(blockIdx.y,
blockDim.y) + threadIdx.y; uint i = __umul24(y, width) + x;
```

```
// Ensure that the thread is within bounds if ((x <
width) && (y < height)) {
```

```
// Alternate between black and red squares in the checkerboard pattern
```

```
// Red for even squares and black for odd squares uchar4 color;
```

```
if (((x / 8) + (y / 8)) % 2 == 0) { // Red square (R = 255,
G = 0, B = 0) color = make_uchar4(0, 0, 255, 255); // Red
```

```
} else {
```

```
// Black square (R = 0, G = 0, B = 0) color =
make_uchar4(0, 0, 0, 255); // Black }
```

```
d_output[i] = color; // Set the pixel color
```

```
}
```

```
}
```

Reflection: I made sure that the thread is in the bounds of the image then for every pixel we check if the X and Y is divisible by 8 or not essentially

splitting the image into 8x8 squares that we then colour black or white and then output the colour.

Exercise: Making a circle

```
__global__ void d_render(uchar4* d_output, uint width, uint height) {    uint x = blockIdx.x *
blockDim.x + threadIdx.x;    uint y = blockIdx.y * blockDim.y + threadIdx.y;

    // Ensure the thread is within the bounds of the image    if (x <
width && y < height) {

        // Normalize pixel coordinates to the range [-1, 1]    float u =
x / (float)width;    float v = y / (float)height;

    u = 2.0 * u - 1.0; // Map to [-1, 1] in x
    v = -(2.0 * v - 1.0); // Map to [-1, 1] in y and flip y-axis

    // Scale u to maintain the aspect ratio of the image    u *=
width / (float)height;

    // Define the radius of the disc in normalized coordinates (in the range [0,
1])
    float radius = 0.2f; // Radius as a fraction of the image size

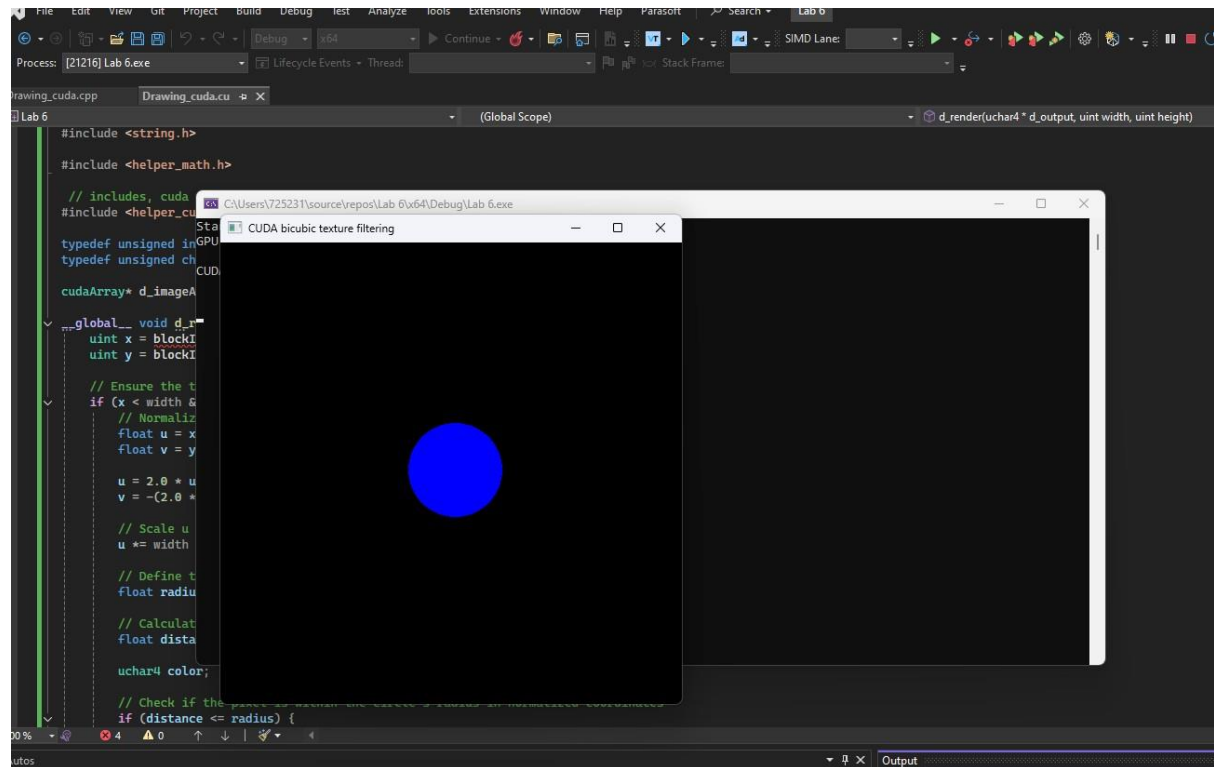
    // Calculate the distance from the center of the image (which is (0, 0) in normalized space)
    float distance = sqrtf(u * u + v * v);

    uchar4 color;

    // Check if the pixel is within the circle's radius in normalized coordinates    if (distance
<= radius) {
        color = make_uchar4(255, 0, 0, 255); // Red color
    }    else
    {
        color = make_uchar4(0, 0, 0, 255); // Black background (outside the disc)
    }

    // Set the pixel color at the correct index    uint i
= y * width + x;    d_output[i] = color;
```

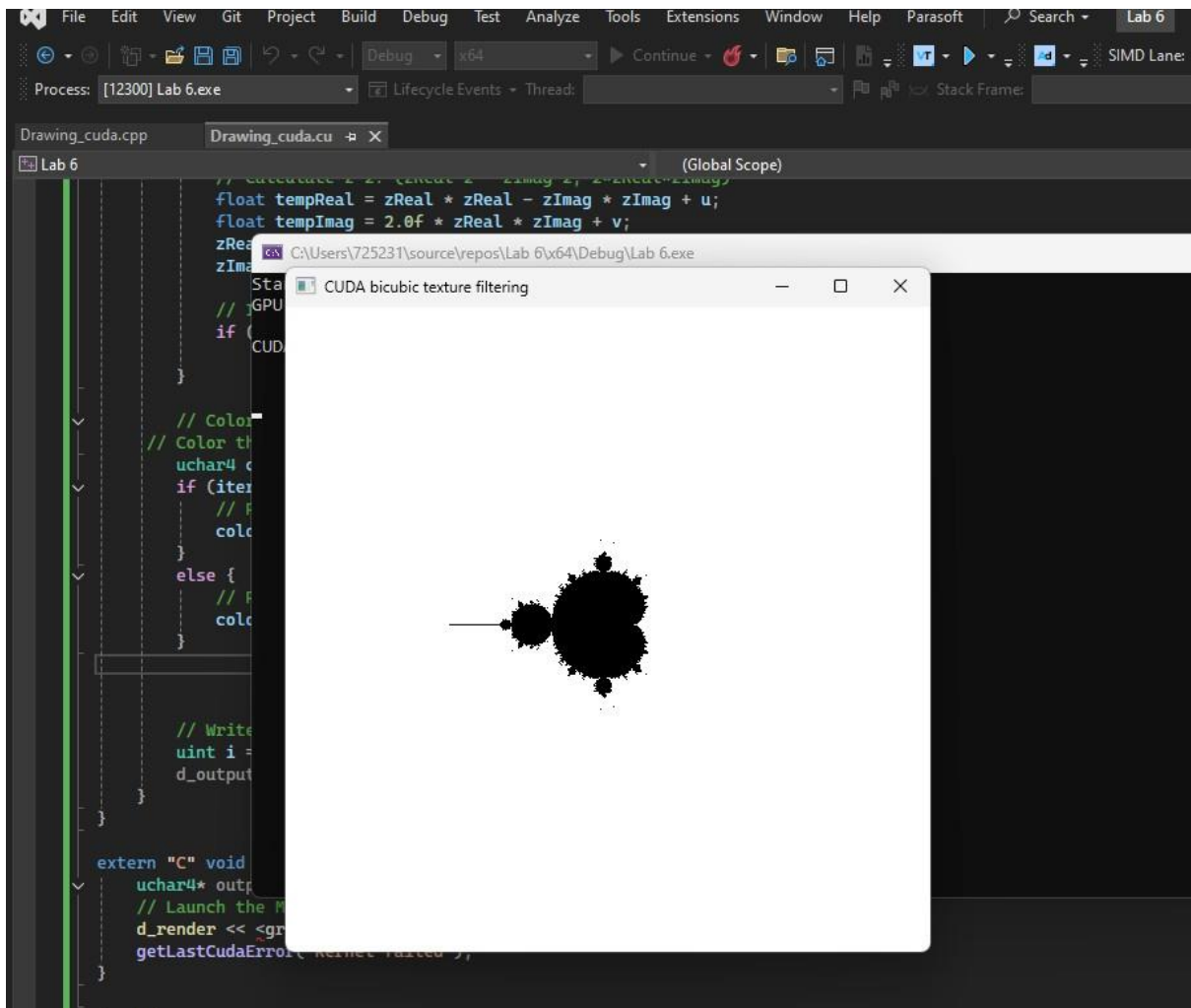
```
}  
  
}
```



Reflection: First you check the pixel is in the bounds of the image and then normalize the coordinates to be between -1 and 1. Then you must adjust the u coordinate to preserve the image's aspect ratio. After that you use Euclidian distance and check the if the distance is less than the radius which is set as a fraction if the image size.

Exercise 3: Drawing the Mandelbrot and Julia Sets.

Mandlebrot



```
Solution: if (x < width && y < height) { //
Normalize pixel coordinates to [0, 1] float u = x
/ (float)width; float v = y / (float)height;

// Map coordinates to [-1, 1] and flip the y-axis u = 2.0f
* u - 1.0f; v = -(2.0f * v - 1.0f);
```

```
// Scale to the larger region [-4, 4] u *=
4.0f; v *= 4.0f;
```

```
// Mandelbrot iteration:
```

```
// For each pixel, interpret (u, v) as the complex constant c = u + i*v.
```

```
// Start with z = 0 and iterate z = z^2 + c.
```

```
float zReal = 0.0f; float
zImag = 0.0f; int iter;
```

```
for (iter = 0; iter < maxIter; iter++) {
```

```

        // Calculate z^2: (zReal^2 - zImag^2, 2*zReal*zImag)    float
tempReal = zReal * zReal - zImag * zImag + u;    float templmag =
2.0f * zReal * zImag + v;    zReal = tempReal;    zImag = templmag;

        // If the magnitude squared of z exceeds 4, the sequence diverges    if (zReal *
zReal + zImag * zImag > 4.0f)    break;

    }

    // Color the pixel based on the iteration count // Color
the pixel based on the iteration count    uchar4 color;
uchar4 color;    if (iter == maxIter) {

        // Points that did not escape are colored black    color =
make_uchar4(0, 0, 0, 255);

    } else {

        // Points that escaped are colored white    color =
make_uchar4(255, 255, 255, 255);

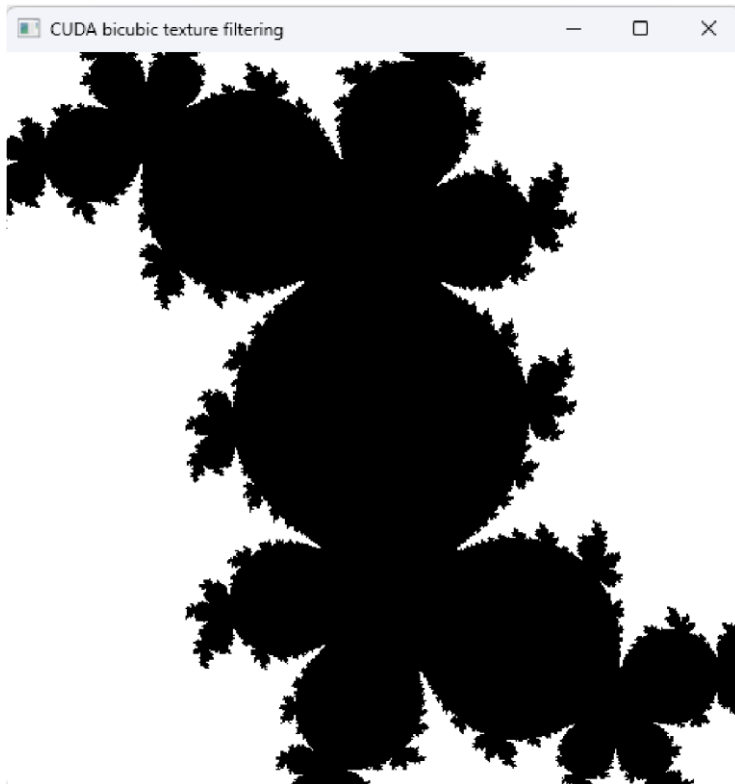
    }

    // Write the computed color to the output array    uint i = y
* width + x;    d_output[i] = color;

```

Reflection: In this task, I modified an existing CUDA texture filtering code to render the Mandelbrot fractal. I learned how to map pixel coordinates to complex numbers and implement the iterative Mandelbrot calculation in a CUDA kernel.

Julia:



Solution:

```

    if (x < width && y < height) {
        // Normalize pixel coordinates to [0, 1]    float
        u = x / (float)width;    float v = y / (float)height;

        // Map coordinates to the range [-1, 1]    u =
        2.0f * u - 1.0f;    v = -(2.0f * v - 1.0f);

        // For a Julia set, use the pixel coordinate (u,v) as the initial z    float
        zReal = u;    float zImag = v;

        int iter;

        // Iterate the function: z = z^2 + c, where c is constant    for (iter = 0;
        iter < maxIter; iter++) {

            // Compute z^2 (in complex form)

            float tempReal = zReal * zReal - zImag * zImag + cReal;    float
            tempImag = 2.0f * zReal * zImag + cImag;    zReal = tempReal;
            zImag = tempImag;

            // If |z|^2 exceeds 4, assume it diverges    if
            (zReal * zReal + zImag * zImag > 4.0f)    break;

```

```

    }

    // Determine color based on iteration count
    uchar4 color;  uchar4 color;  if (iter == maxIter) {

        // Points that did not escape are colored black    color =
        make_uchar4(0, 0, 0, 255);

    } else {

        // Points that escaped are colored white    color =
        make_uchar4(255, 255, 255, 255);

    }

    // Write the pixel color to the output image    uint i =
    y * width + x;    d_output[i] = color;

    }

}

```

Reflection

In this task, I extended my fractal rendering project by modifying the CUDA kernel to generate a Julia set. I learned how to adapt the Mandelbrot implementation by using each pixel's coordinate as the initial value for z and applying a fixed complex constant c (e.g., $0.25 + 0.5i$) to obtain the Julia set.

Final Lab Cuda

Bird. H File

This header file defines the Bird struct used to represent individual birds in a CUDA-accelerated flocking simulation. Each Bird instance holds the essential attributes required to simulate flocking behaviour in 3D space. The struct is aligned to 16 bytes using `__align__(16)` to ensure compatibility and optimal memory access patterns when used in CUDA kernels. Each Bird occupies exactly 40 bytes, making it suitable for efficient memory transfer and access on the GPU.

Flocking. H File

This header file defines the core behavioural rules used in the CUDA-accelerated flocking simulation: separation, alignment, and cohesion. Each rule is implemented as a `__device__` function, allowing them to be executed in parallel on the GPU for every bird in the simulation. The separation rule computes a steering force that pushes a bird away from nearby flock mates to avoid overcrowding, using an inverse distance weighting to generate stronger repulsion from closer neighbours. The alignment rule steers the bird to match the average heading of nearby birds by averaging their velocities, promoting synchronized group movement. The cohesion rule calculates a steering force that moves the bird toward the average position of its neighbours, helping the flock stay together as a unit. All three rules depend on proximity checks within defined distances and make use of vector math (via float3 operations) to produce normalized direction vectors. These behaviours are fundamental to realistic flocking motion and are combined later in the simulation to compute each bird's new acceleration.

Bird Update. H File

This header file defines the logic used to update each bird's position and velocity based on flocking behaviours in a CUDA-accelerated simulation. The update function is a `__device__` function that runs on the GPU and calculates the next state of an individual bird using the separation, alignment, and cohesion rules defined in flocking. These behaviours are weighted by their proximity to other birds, with normalized distance thresholds based on the screen width to maintain consistency across different resolutions. The resulting steering forces are combined into a single acceleration vector, which is then clamped to a maximum value to maintain realistic movement. This acceleration is applied to the bird's velocity, which is also constrained by a maximum speed to avoid erratic behaviour. The bird's position is updated based on this velocity and the given time step (delta Time). Boundary conditions are applied to wrap birds around the screen edges, ensuring continuous movement within a 2D space normalized

to a range of 0.0 to 1.0. This update function enables real-time simulation of natural flocking behaviour in large groups of agents running in parallel on the GPU.

Drawing Cuda Cpp File

This CUDA C++ file handles the rendering of birds in the flocking simulation using OpenGL interoperability. It maps an OpenGL Vertex Buffer Object (VBO) to CUDA memory, allowing bird position and colour data to be directly written on the GPU without transferring it back to the CPU. The render Birds kernel populates the VBO with vertex positions and colours for each bird, constructing simple triangle representations oriented based on the bird's velocity. The rendering respects normalized screen coordinates, ensuring consistent visualization across different resolutions. This efficient GPU-based rendering pipeline enables real-time visualization of thousands of flocking agents.

Drawing Cuda CU File

This CUDA source file implements the simulation and rendering logic for a GPU-accelerated flocking system. It features the updateBirdsKernel, which updates each bird's position and velocity based on flocking behaviours using a tiled approach that leverages shared memory to store local bird data during each iteration. This significantly reduces expensive global memory accesses when comparing each bird to its neighbours, improving performance and scalability for large flocks. After computing acceleration from separation, alignment, and cohesion forces, the updated velocities and positions are written back to global memory. The d_render kernel then draws each bird onto a 2D output buffer by mapping their normalized positions to pixel coordinates. Additional functionality includes timing the update kernel to monitor performance, and a debug kernel to print bird data. This file demonstrates an efficient use of shared vs. global memory to optimize parallel updates and real-time rendering of flocking behaviour on the GPU.

Performance:

Originally, I made the model with global memory and each bird had its own thread however global memory is not as good compared to shared memory as it has longer latency and is slower. So, I made a second model that uses shared memory and then I timed how long it took each kernel to complete and made an average after 1000 frames. Each kernel had to deal with 50 birds.

Shared memory: Average update kernel time over 1020 frames: 3.021 ms

Global memory: Average update kernel time over 1020 frames: 3.599 ms

This clearly shows that the model with shared memory is better than the model without as its kernel finishes 0.578ms faster than global memory.